Yanwei Zheng, Shandong University

Exercise Library

for Compiling Principles and Techniques

Author: Yanwei Zheng

Institute: Shandong University

Date: April 6, 2024

Version: 1.0

山东大学

Contents

1 Introduction to Compilers 1
Chapter 1 Exercise 1

2 Grammar and Language Design 4
Chapter 2 Exercise 4

3 lexical analysis 7
Chapter 3 Exercise 7

4 Syntax Analysis 14
Chapter 4 Exercise 14

5 Symbol Table 24
Chapter 5 Exercise 24

6 Runtime Storage Organization 26
Chapter 6 Exercise 26

7 Grammar Guided Translation and Intermediate Code Generation 30
Chapter 7 Exercise 30

8 35
Chapter 8 Exercise 35

9 Target Code Generation 42
Chapter 9 Exercise 42

10 Translation of Object Oriented Languages 48
Chapter 10 Exercise 48

Yanwei Zheng, Shandong University

“Construction of Exercise Library for Compiler Course” by the “Virtual Teaching and Research Office for Compiler Course” (2023-003)

Supporting Textbook: Yanwei Zheng, Dongxiao Yu, and Jin Zhou. Compiler Principles and Techniques. Beijing: Tsinghua University Press, 2025.

121 algorithms and 125 translation schemes, constructing a compiler from scratch.

The paper version contains 894k words, and the electronic version adds up to 954k words.

Yanwei Zheng, Shandong University

Figure 1 Supporting Textbook

Yanwei Zheng
April 15, 2026

Contents

Yanwei Zheng, Shandong University

Chapter 1 Introduction to Compilers

Chapter 1 Exercise

1. Briefly describe the difference between natural language and formal language.
2. Based on the relationship to the hardware, what categories can programming languages be divided into?
3. Explain what is a translator, compiler, interpreter, and assembler.
4. List the compiled languages and interpreted languages you know.
5. Explain what is run-time and compile-time.
6. Explain what is a host machine and a target machine. Li Hua is developing a program on Machine A, which has the Windows operating system installed, to run on a HarmonyOS phone, Machine B. Which machine is the host and which is the target?
7. Li Hua designed a new language named Yi Language and intended to implement it using a C language compiler on an x86 machine with a Windows operating system. Please describe the implementation process and illustrate it with a T-diagram.
8. There is a C language compiler for x86 machines and Windows operating systems. Li Hua attempts to port it to the HarmonyOS operating system. Explain the implementation process and draw a T-diagram to represent it.
9. Li Hua designed a new language, named Yi Language, and intended to implement it on x86 machines and Windows operating systems using a bootstrap (self-compilation) approach. Please describe the implementation process.
10. From the perspective of language paradigms, explain the classification of high-level programming languages.
11. List the imperative languages and declarative languages you know.
12. List common high-level programming language structures.
13. What are the main features of procedural-oriented languages and object-oriented languages?
14. What is the main difference between object-based languages and object-oriented languages?
15. What are the main features of dynamic languages?
16. What are the main features of a language with nested procedure definitions?
17. Investigate the concept of program closures and provide an explanation. List several programming languages that support program closures.
18. What is byte alignment? Why is byte alignment necessary?
19. Assuming space allocation starts from address 1000h, please write the allocated addresses for each variable in the following C-style declaration statement: "int i1; bool b1; bool b2; int i2; short s1; int i3;".
20. List at least six common sentence forms in high-level languages.
21. What are the forms of target code?
22. List common CPU architectures, specify the instruction sets they adopt, and provide examples of manufacturers utilizing these architectures.
23. Draw a figure and explain the division of storage space during runtime.
24. List all the general-purpose registers of x86.
25. What are the advantages of the LEA instruction over the MOV instruction when performing

Yanwei Zheng, Shandong University

address calculation?

26. Write the following instructions.
    (1) Load the address 0ABCDh into the register EAX.
    (2) Load the address of the variable x in the static data area into the register EAX.
27. Write an assembly program.
    (1) Declare three 32-bit variables a, b, and c in the static data area, where a and b are initialized to 10 and 20 respectively, and c is not initialized.
    (2) Write code to store the result of a + b in the variable c.
28. Write an assembly program.
    (1) Declare three float variables a, b, and c in the static data area, where a and b are initialized to 3.14 and 2.0 respectively, and c is not initialized.
    (2) Write code to save the result of a*b to the variable c.
29. Write an assembly program.
    (1) Declare two float variables a and b, and one 32-bit integer variable c in the static data area, where a and b are initialized to 3.14 and 2.0 respectively, and c is not initialized.
    (2) Write code to round the result of a*b and store it in variable c.
30. Calculate a/b, where a and b are both unsigned integer variables in the static data area. Save the quotient and remainder to the ESI and EDI registers, respectively. Write the assembly code.
31. Calculate a/b, where a and b are both signed integer variables in the static data area. Save the quotient and remainder to the ESI and EDI registers, respectively. Write the assembly code.
32. Write an assembly process to implement the addition of 1 to a number n, and store the result in result, where n and result have been declared and initialized in the static data area. It is required to protect the registers used.
33. Write an assembly process to implement the factorial of a number n, with the result stored in result. Both n and result have been declared and initialized in the static data area. It is required to protect the registers used.
34. In the static data area, two real8 variables, x and y, are declared and initialized. Additionally, a byte variable named result is declared. The comparison x > y is performed, and if true, result is set to 1; otherwise, it is set to 0. Write the assembly process to implement this procedure.
35. In the static data area, three real8 variables, a, b, and c, are declared, which are the coefficients of the equation $ax^2+bx+c=0$ and have been initialized. Additionally, one byte variable hasRoot, as well as two real8 variables x1 and x2, are declared but not initialized. If the equation has solutions, hasRoot is set to 1, and the two roots are written to x1 and x2; if the equation has no solutions, hasRoot is set to 0. Write the assembly process to implement this procedure.
36. Multiplication instructions are relatively time-consuming, with the execution time of a single multiplication instruction typically being several times to over ten times that of an addition instruction. For the multiplication instruction "imul eax, 70", the same result can be achieved using shift instructions such as shl, as well as mov and add/sub instructions. Write the assembly process to implement this procedure.
37. In the static data section, three real8 variables, denoted as a, b, and c, are declared and initialized, representing the coefficients of the function $f(x) = ax^2 + bx + c$ (where $a > 0$). Additionally, one real8 variable, denoted as yMin, is declared but not initialized. Write an assembly program to find the minimum value of the function $f(x)$ using Newton's method and store the result in

Yanwei Zheng, Shandong University

yMin.

38. In the static data section, three real8 variables, denoted as a, b, and c, are declared and initialized, representing the coefficients of the function $f(x) = ax^2 + bx + c$ (where $a > 0$). Additionally, one real8 variable, denoted as yMin, is declared but not initialized. Write an assembly program to find the minimum value of the function $f(x)$ using gradient descent method and store the result in yMin.
39. In the static data area, three real8 variables a, b, and c are declared, which are the coefficients of the function $f(x) = ax^2 + bx + c$ ($a \neq 0$). They have been initialized. Additionally, two real8 variables, xExtremum and yExtremum, are declared but not initialized. Write an assembly program to find the extremum of the function $f(x)$ using Newton's method, and write the x and y values at which the extremum is obtained into xExtremum and yExtremum respectively (Hint: If $a > 0$, find the minimum value of $f(x)$; if $a < 0$, find the maximum value of $f(x)$, which is equivalent to finding the minimum value of $-f(x)$).
40. In the static data area, three real8 variables a, b, and c are declared, which are the coefficients of the function $f(x) = ax^2 + bx + c$ ($a \neq 0$). They have been initialized. Additionally, two real8 variables, xExtremum and yExtremum, are declared but not initialized. Write an assembly program to find the extremum of the function $f(x)$ using gradient descent method, and write the x and y values at which the extremum is obtained into xExtremum and yExtremum respectively (Hint: If $a > 0$, find the minimum value of $f(x)$; if $a < 0$, find the maximum value of $f(x)$, which is equivalent to finding the minimum value of $-f(x)$).
41. Draw a figure to represent the classification of intermediate languages.
42. Find the postfix expression of the expression "$(a + b) * (a - b)$".
43. Find the postfix expression of the expression "$(a + b)\hat{}(a - b)\hat{}c$".
44. Find the postfix expression of the expression "$a \vee b \neq c + d \wedge a * b < f$".
45. Write the quadruple sequence for the assignment statement "$x = a + b + (a + b) * (a - b)$".
46. Write the triplet sequence for the assignment statement "$x = a + b + (a + b) * (a - b)$".
47. Write the indirect triplet sequence for the assignment statement "$x = a + b + (a + b) * (a - b)$".
48. Write the quadruple sequence for the assignment statement "$x = (a + b)\hat{}(a - b)\hat{}(a + b)$".
49. Write the triplet sequence for the assignment statement "$x = (a + b)\hat{}(a - b)\hat{}(a + b)$".
50. Write the indirect triplet sequence for the assignment statement "$x = (a + b)\hat{}(a - b)\hat{}(a + b)$".
51. Write the quadruple sequence for the assignment statement "$x = a \vee b \neq c + d \wedge a * b < c + d$".
52. Write the triplet sequence for the assignment statement "$x = a \vee b \neq c + d \wedge a * b < c + d$".
53. Write the indirect triplet sequence for the assignment statement "$x = a \vee b \neq c + d \wedge a * b < c + d$".
54. Draw a figure to represent the composition of the compiler, and explain the functions of each part.
55. Explain the front-end and back-end of a compiler.

Yanwei Zheng, Shandong University

Chapter 2 Grammar and Language Design

Chapter 2 Exercise

1. In the lexical and syntax analysis stages, what do symbols and symbol strings represent respectively?
2. Let $x = 00$ and $y = 11$ be strings over the alphabet $\Sigma = \{0, 1\}$. Write the results of the following expressions: xy, yxy, x^3, x^2y^2, $|yx|$.
3. Let the symbol string $x = 00, y = 11$ on $Sigma = 0, 1$, and write the prefix and suffix of xy.
4. Let the symbol string set $X = \{0, 11\}, Y = \{01, 10\}$ on $\Sigma = \{0, 1\}$, write XY, YX, X^2, $X\{\varepsilon\}$.
5. Let $x = 001011$.
   (1) Write the set U consisting of prefixes with a length of no more than 3 for x.
   (2) Write the set V consisting of suffixes with a length of no more than 3 for x.
   (3) Find the values of UV and VU.
   (4) Find the values of U^2 and V^2.
6. Given the grammar $G[E] : E \to EAE|i, A \to +| - | * |/$.
   (1) Using standard deduction, derive the sentence $i+i*i$ from the start symbol of the grammar.
   (2) Using standard reduction, reduce the sentence $i - i/i$ to the start symbol of the grammar.
   (3) Write the language represented by the grammar.
   (4) Determine whether the grammar is ambiguous and provide reasons.
7. Given the grammar $G[E] : E \to E \wedge E|E \vee E|(E)|i$, where i represents any boolean variable.
   (1) Using standard deduction, derive the sentence $i\ edge(i\ veei)$ from the start symbol of the grammar.
   (2) Using standard reduction, reduce the sentence $i \vee (i \wedge i)$ to the start symbol of the grammar.
   (3) Write the language represented by the grammar.
   (4) Determine whether the grammar is ambiguous and provide reasons.
8. Given the grammar $G[E] : E \to E + T|T, T \to T * F|F, F \to (E)|i$.
   (1) Write the phrases, immediate phrases, and handle of sentential form $i * (E)$.
   (2) Write the phrases, immediate phrases, and handle of sentential form $i * (i + i)$.
9. Given the grammar $G[E] : E \to E \vee T|T, T \to T \wedge F|F, F \to \neg F|(E)|i$.
   (1) Write the phrases, immediate phrases, and handle of sentential form $i \wedge (E)$.
   (2) Write the phrases, immediate phrases, and handle of sentential form $i \wedge \neg(i \vee i)$.
10. Given the grammar $G[E] : E \to (A, E, E)|i, A \to +| - | * |/$.
    (1) The Lisp-like code (θ, a, b) represents the result of $a\theta b$, where θ is an operator. Write the Lisp-like program form for $a * (b + c)/d$.
    (2) The i represents any variable. Try to derive the above sentence from the start symbol using standard derivation;
    (3) Reduce the above sentence to the start symbol using canonical reduction.
    (4) Draw the syntax tree of the above sentence.
    (5) Write the phrases, immediate phrases, and handle of the above sentence.
11. Given the grammar $G[S] : S \to AB, A \to Aa|b, B \to bB|a$, determine the language defined by this grammar.
12. Given the grammar $G[S] : S \to (S)|i$, determine the language defined by this grammar.

Yanwei Zheng, Shandong University

13. Given the grammar $G[S] : S \to (S)|\varepsilon$, determine the language defined by this grammar.
14. Construct a grammar such that the language it describes is the set of positive even numbers.
15. What are the production forms of context-dependent grammar and context-free grammar respectively? What is the main difference between them?
16. Construct a regular expression to represent "binary numbers divisible by 4", and write its corresponding context-free grammar.
17. Construct a regular expression to represent "decimal numbers divisible by 4", and write its corresponding context-free grammar.
18. Write the context-free grammar for the regular expressions α^*, $\alpha\beta^*$, and $\alpha^*\beta$ respectively.
19. $\Sigma = \{0, 1\}$, construct a regular expression that represents words with odd lengths, where 0 and 1 appear alternately, such as 0, 1, 010, 101, 01010, 10101, $\cdots$. And design a context-free grammar to represent this regular expression.
20. Design a context-free grammar to represent the regular expression $(00|11|(01|10)^*)^*$.
21. If the array element format is $id[num][num]\cdots[num]$, where id is the array name and num is the subscript, try to write its regular expression and construct the corresponding context-free grammar.
22. Consider modifying the C-style function definition in the form of "int Fun(int a, b; float x, y)", where the function returns an int type and has two int and two float parameters. Considering only int and float data types, the function can have no return value. Write the formal grammar for this function definition and construct its corresponding context-free grammar.
23. Assuming there is a LISP-like language, whose form is (θ, E, E), where θ is an operator, and E is either an operand i or an expression of the form (θ, E, E), construct its grammar.
24. Design a disambiguous grammar that enables the representation of logical expressions, where all operands involved in the logical expressions are Boolean variables, represented by the terminal symbol i.
25. Design a unambiguous grammar that enables the representation of logical expressions. Logical expressions involve operands that are either Boolean variables i or relational operations of the form $i\theta i$. In these relational operations, θ is a relational operator, and i is an integer or real variable.
26. Design a unambiguous grammar that enables the representation of logical expressions. The operands involved in logical expressions are either Boolean variables i or relational operations of the form $E\theta E$; in the relational operations, θ is a relational operator, and E is an arithmetic expression, considering only addition, multiplication, and parentheses operations.
27. Given the grammar $G = (\{S, A, B\}, \{i, x, (,)\}, P, S)$, where $P = \{S \to (S)|i|A, A \to xAx, B \to xBx|i\}$. Try to eliminate useless productions.
28. Given the grammar $G = (\{S, A, B\}, \{i, x, (,)\}, P, S)$ $P = \{S \to (S)|i|AB, A \to xAx, B \to xBx|i\}$. Try to eliminate useless productions.
29. Given the grammar $G = (\{S, A, B\}, \{o, s, (,)\}, P, S)$ $P = \{S \to (A), A \to S|B, B \to oBo|s\}$. Try to eliminate single-V_N productions.
30. Given the grammar $G = (\{S, A, B\}, \{o, s, (,)\}, P, S)$ $P = \{S \to (A)|A, A \to S|B, B \to oBo|s\}$. Try to eliminate single-V_N productions.
31. Given the grammar $G = (\{S, A, B\}, \{o, s, (,)\}, P, S)$ $P = \{S \to (A), A \to S|B, B \to oBs|\varepsilon\}$. Try to eliminate or normalize the production of null symbols.

Yanwei Zheng, Shandong University

32. Given the grammar $G = (\{S, A, B\}, \{o, s, (,)\}, P, S)$ $P = \{S \to (A)|A, A \to S|B, B \to oBs|\varepsilon\}$. Try to eliminate or normalize the production of null symbols.

Yanwei Zheng, Shandong University

Chapter 3 lexical analysis

Chapter 3 Exercise

1. What is the task of lexical analysis?
2. What are the categories of tokens?
3. How are tokens classified and encoded?
4. Briefly describe two implementation forms of lexical analyzers and their advantages and disadvantages.
5. Briefly describe the function of the filtering stage in lexical analysis. Can it be merged with the scanning function?
6. Explain the significance and implementation method of the double-buffering strategy.
7. What is lookahead search? Give an example.
8. Design a DFA M over $\Sigma = \{0, 1, \cdots, 9\}$ such that it accepts decimal numbers divisible by 3.
9. Design a DFA M over $\Sigma = \{0, 1\}$ such that it accepts binary numbers divisible by 3.
10. Design a DFA M over $\Sigma = \{0, 1\}$ such that it can recognize lexemes containing an even number of 0s and an even number of 1s.
11. Design a DFA M over $\Sigma = \{0, 1, \cdots, 9\}$ that accepts decimal numbers divisible by 4.
12. Design a DFA M over $\Sigma = \{0, 1\}$ such that it accepts binary numbers divisible by 4.
13. Design a DFA M that can recognize comments in C language.
14. Design a DFA M that can recognize hexadecimal digits in C language.
15. Design a DFA M that can recognize all forms of float numbers (excluding positive and negative signs) in C language.
16. Write a program for preprocessing C language source code to remove comments.
17. Given a regular expression $aa(a|b)^*bb$.
    (1) Construct an NFA and make its each edge be labeled by single symbol or ε.
    (2) Convert the NFA to DFA.
    (3) Simplify the DFA.
18. Given a regular expression $b(aa)^*ba(bb)^*a$.
    (1) Construct an NFA and make its each edge be labeled by single symbol or ε.
    (2) Convert the NFA to DFA.
    (3) Simplify the DFA.
19. Given a regular expression $(0|1)^*(01|10)(0|1)^*$.
    (1) Construct an NFA and make its each edge be labeled by single symbol or ε.
    (2) Convert the NFA to DFA.
    (3) Simplify the DFA.
20. Given a regular expression $(a|b)(a|b|0|1)^*$.
    (1) Construct an NFA and make its each edge be labeled by single symbol or ε.
    (2) Convert the NFA to DFA.
    (3) Simplify the DFA.
21. Given a regular expression $(0|1)^+.(0|1)^+$.
    (1) Construct an NFA and make its each edge be labeled by single symbol or ε.
    (2) Convert the NFA to DFA.

Yanwei Zheng, Shandong University

(3) Simplify the DFA.

22. The regular expression for C-style variable declaration is $(i|r)v(,v)^*$, where i and r represent integer and real data types respectively, and v represents the variable name.
    (1) Construct an NFA and make its each edge be labeled by single symbol or ε.
    (2) Convert the NFA to DFA.
    (3) Simplify the DFA.
23. The regular expression for Pascal-style variable declaration is $v(,v)^* : (i|r)$, where i and r represent integer and real data types respectively, and v represents the variable name.
    (1) Construct an NFA and make its each edge be labeled by single symbol or ε.
    (2) Convert the NFA to DFA.
    (3) Simplify the DFA.
24. The regular expression for array element referencing is: $v[(v,)^*v]$; where v represents a variable or array name.
    (1) Construct an NFA and make its each edge be labeled by single symbol or ε.
    (2) Convert the NFA to DFA.
    (3) Simplify the DFA.
25. The regular expression for C-style procedure calls is: $v\backslash(v(,v)^*|\varepsilon\backslash)$; where v represents a variable or procedure name, and \ represents an escape character symbol, meaning that \(and \) represent the occurrence of a left parenthesis and a right parenthesis in the regular expression, respectively, and these two bracket symbols are not meta-symbols in the regular expression.
    (1) Construct an NFA and make its each edge be labeled by single symbol or ε.
    (2) Convert the NFA to DFA.
    (3) Simplify the DFA.
26. The grammar $G[S]$ for C-style variable declarations has the following production rules, where i and r represent integer and real data types respectively, and v represents a variable name. Construct a finite automaton (FA) equivalent to it.

    $S \to iV$ $\quad S \to rV$ $\quad V \to vL$ $\quad L \to ,V$ $\quad L \to \varepsilon$

27. The grammar $G[S]$ for C-style variable declarations has the following production rules, where i and r represent integer and real data types, respectively, and v represents a variable name. Construct an equivalent finite automaton (FA) for it.

    $S \to Tv$ $\quad S \to Av$ $\quad A \to S,$ $\quad T \to i$ $\quad T \to r$

28. The grammar $G[S]$ for Pascal-style variable declarations has the following production rules, where i and r represent integer and real data types, respectively, and v represents a variable name. Construct a finite automaton (FA) equivalent to it.

    $S \to vA$ $\quad A \to ,B$ $\quad B \to vA$
    $A \to : T$ $\quad T \to i$ $\quad T \to r$

29. The grammar $G[S]$ for Pascal-style variable declarations has the following production rules, where i and r represent integer and real data types, respectively, and v represents a variable name. Construct an equivalent finite automaton (FA) for it.

    $S \to Ai$ $\quad S \to Ar$ $\quad A \to B :$
    $B \to v$ $\quad B \to Uv$ $\quad U \to B,$

Yanwei Zhang, Shandong University

30. The grammar $G[S]$ representing array element references has the following production rules, where v denotes a variable name or array name. Construct a finite automaton (FA) equivalent to it.

$S \to vA$ $\quad A \to [L$ $\quad L \to vV$

$L \to vR$ $\quad V \to ,L$ $\quad R \to]$

31. The grammar $G[S]$ representing array element references has the following production rules, where v denotes a variable name or array name. Construct an equivalent finite automaton (FA) for it.

$S \to R]$ $\quad R \to Lv$ $\quad L \to R,$ $\quad L \to A[$ $\quad A \to v$

32. The grammar $G[S]$ for C-style procedure calls has the following production rules, where v represents a variable name or procedure name. Construct an equivalent finite automaton (FA) for it.

$S \to vA$ $\quad A \to (L$ $\quad A \to (R$ $\quad L \to vV$

$L \to vR$ $\quad V \to ,L$ $\quad R \to)$

33. The grammar $G[S]$ for C-style procedure calls has the following production rules, where v represents a variable name or procedure name. Construct an equivalent finite automaton (FA) for it.

$S \to R)$ $\quad S \to B)$ $\quad R \to Lv$ $\quad L \to R,$

$L \to A($ $\quad A \to v$ $\quad B \to A($

34. Convert the FA represented in Figure 3.1 into a right-linear grammar G_R.
35. Convert the FA represented in Figure 3.1 into a left-linear grammar G_L.

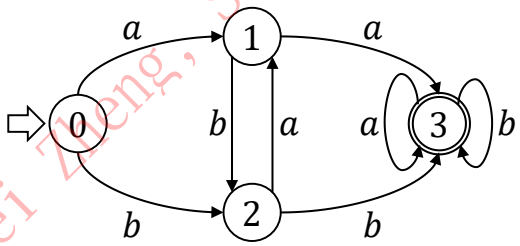


Figure 3.1 DFA

Yanwei Zheng, Shandong University

36. Convert the FA represented in Figure 3.2 into a right-linear grammar G_R.
37. Convert the FA represented in Figure 3.2 into a left-linear grammar G_L.

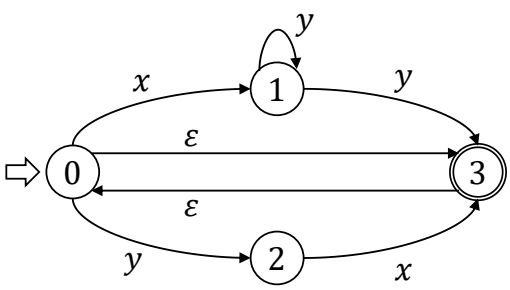


Figure 3.2 FA

38. Convert the FA represented in Figure 3.3 into a right-linear grammar G_R.
39. Convert the FA represented in Figure 3.3 into a left-linear grammar G_L.

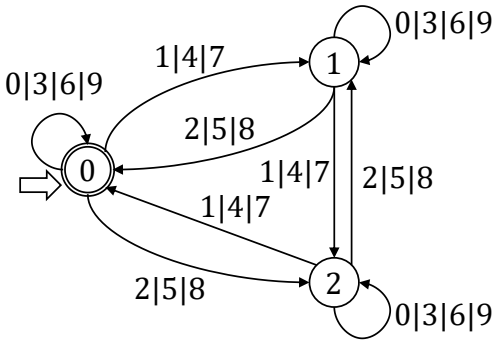


Figure 3.3 FA

40. Convert the FA represented in Figure 3.4 into a right-linear grammar G_R.
41. Convert the FA represented in Figure 3.4 into a left-linear grammar G_L.

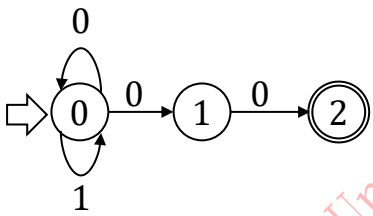


Figure 3.4 FA

Yanwei Zheng, Shandong University

42. Convert the FA represented in Table 3.9 into a right-linear grammar G_R.
43. Convert the FA represented in Table 3.9 into a left-linear grammar G_L.

Table 3.9 DFA that accepts decimal numbers divisible by 4

	0|4|8	1|5|9	2|6	3|7
0	0	1	2	3
1	2	3	0	1
2	0	1	2	3
3	2	3	0	1

44. Convert the regular expression $(00|11|(01|10)^*)^*$ into a right-linear grammar G_R.
45. Convert the regular expression $(00|11|(01|10)^*)^*$ into a left-linear grammar G_L.
46. Convert the regular expression $b(aa)^*(bb)^*a$ into a right-linear grammar G_R.
47. Convert the regular expression $b(aa)^*(bb)^*a$ into a left-linear grammar G_L.
48. The regular expression for C-style variable declaration is: $(i|r)v(,v)^*$; where i and r represent integer and real data types respectively, and v represents the variable name. Convert it into a right-linear grammar G_R.
49. The regular expression for C-style variable declaration is: $(i|r)v(,v)^*$; where i and r represent integer and real data types respectively, and v represents the variable name. Convert it into a left-linear grammar G_L.
50. The regular expression for Pascal-style variable declarations is: $v(,v)^* : (i|r)$; where i and r represent integer and real data types respectively, and v represents the variable name. Convert it into a right-linear grammar G_R.
51. The regular expression for Pascal-style variable declarations is: $v(,v)^* : (i|r)$; where i and r represent integer and real data types respectively, and v represents the variable name. Convert

it into a left-linear grammar G_L.

52. The regular expression for array element reference is: $v[v(,v)^*]$; where v represents a variable name or array name, which is converted into a right-linear grammar G_R.
53. The regular expression for array element reference is: $v[v(,v)^*]$; where v represents a variable name or array name, which is converted into a left-linear grammar G_L.
54. The regular expression for C-style procedure calls is: $v\backslash(v(,v)^*|\varepsilon\backslash)$; where v represents a variable name or procedure name, and \ represents an escape character symbol, meaning that \(and \) represent the occurrence of a left parenthesis and a right parenthesis in the regular expression, respectively. These two bracket symbols are not meta-symbols in the regular expression and are converted into a right-linear grammar G_R.
55. The regular expression for C-style procedure calls is: $v\backslash(v(,v)^*|\varepsilon\backslash)$; where v represents a variable name or procedure name, and \ represents an escape character symbol, meaning that \(and \) represent the occurrence of a left parenthesis and a right parenthesis in the regular expression, respectively. These two bracket symbols are not meta-symbols in the regular expression and are converted into a left-linear grammar G_L.
56. The grammar $G[S]$ for C-style variable declarations has the following production rules, where i and r represent integer and real data types, respectively, and v represents a variable name. Construct an equivalent regular expression for it.

    $S \to iV$ $\quad S \to rV$ $\quad V \to vL$ $\quad L \to ,V$ $\quad L \to \varepsilon$

57. The grammar $G[S]$ for C-style variable declarations has the following production rules, where i and r represent integer and real data types, respectively, and v represents a variable name. Construct an equivalent regular expression for it.

    $S \to Tv$ $\quad S \to Av$ $\quad A \to S,$ $\quad T \to i$ $\quad T \to r$

58. The grammar $G[S]$ for Pascal-style variable declarations has the following production rules, where i and r represent integer and real data types, respectively, and v represents a variable name. Construct an equivalent regular expression for it.

    $S \to vA$ $\quad A \to ,B$ $\quad B \to vA$
    $A \to :T$ $\quad T \to i$ $\quad T \to r$

59. The grammar $G[S]$ for Pascal-style variable declarations has the following production rules, where i and r represent integer and real data types, respectively, and v represents a variable name. Construct an equivalent regular expression for it.

    $S \to Ai$ $\quad S \to Ar$ $\quad A \to B:$
    $B \to v$ $\quad B \to Uv$ $\quad U \to B,$

60. The grammar $G[S]$ representing array element references has the following production rules, where v denotes a variable name or array name. Construct an equivalent regular expression for it.

    $S \to vA$ $\quad A \to [L$ $\quad L \to vV$
    $L \to vR$ $\quad V \to ,L$ $\quad R \to]$

61. The grammar $G[S]$ representing array element references has the following production rules, where v denotes a variable name or array name. Construct an equivalent regular expression for it.

Yanwei Zheng, Shandong University

$S \to R]$ $\quad R \to Lv$ $\quad L \to R,$ $\quad L \to A[$ $\quad A \to v$

62. The grammar $G[S]$ for C-style procedure calls has the following production rules, where v represents a variable name or procedure name. Construct an equivalent regular expression for it.

$S \to vA$ $\quad A \to (L$ $\quad A \to (R$ $\quad L \to vV$
$L \to vR$ $\quad V \to ,L$ $\quad R \to)$

63. The grammar $G[S]$ for C-style procedure calls has the following production rules, where v represents a variable name or procedure name. Construct its equivalent regular expression.

$S \to R)$ $\quad S \to B)$ $\quad R \to Lv$ $\quad L \to R,$
$L \to A($ $\quad A \to v$ $\quad B \to A($

64. Convert the FA represented in Figure 3.1 into a regular expression.
65. Convert the FA represented in Figure 3.2 into a regular expression.
66. Convert the FA represented in Figure 3.3 into a regular expression.
67. Convert the FA represented in Figure 3.4 into a regular expression.
68. Convert the FA represented in Table 3.9 into a regular expression.
69. Write out the 2-tuple lexeme sequence obtained after lexical analysis of code 3.1.

Code 3.1 factorial calculation

```
/* factorial calculation **********
** input n, output n! ****/
int factorial(int n) {
  // if n is 1, return 1
  if (n == 1)
    return 1;
  // else, recursive computation
  return n * factorial(n - 1);
}
```

70. According to the C language standard, comments are replaced with whitespace characters, and multiple consecutive whitespace characters are merged into one whitespace character. Based on this rule, write the 2-tuple lexeme sequence obtained after lexical analysis of code 3.2.

Code 3.2 Calculate factorial in C language

```
int factorial(int/*formal parameter*/n) {
  if (n =/*Whether n is 1*/= 1)
  return 1;
  return n * fact/*recursive call*/orial(n - 1);
}
```

71. In lexical analysis, if the comment is directly deleted, write the code ref lst: C3Excise2 to obtain the 2-tuple lexeme sequence after lexical analysis.
72. Write the code ref lst: C3Excise3 to obtain the the 2-tuple lexeme sequence after lexical analysis.

Code 3.3 Calculate maximum value

```
int max(int m, int n) {
```

Yanwei Zheng, Shandong University

```
    return m > n ? m : n;
}
int mian() {
    int inputX;
    scanf("%d", &inputX);
    printf("max = %d\r\n", max(inputX, -1));
}
```

Yanwei Zheng, Shandong University

Chapter 4 Syntax Analysis

Chapter 4 Exercise

1. What are the two main categories of syntax analysis based on the direction of analysis? What are the typical analysis methods for each?
2. What are the meanings of each letter and number in LL(1) analysis method?
3. What is backtesting in syntax analysis? What are its drawbacks?
4. Before conducting top-down analysis, it is necessary to preprocess the grammar to ensure that it meets what requirements?
5. What are the hazards of left recursion and left common factor in top-down analysis?
6. What are the conditions for conducting top-down LL(1) analysis?
7. Write the formula that eliminates left recursive.
8. Write the definitions of $First(\alpha)$, $First(A)$, and $Follow(A)$ in LL(1) analysis, where α is the right part of any production equation (alternative) and A is any non terminal symbol.
9. Given grammar $G[S]$, and the production rules are as follows. Eliminate its left recursion.

   $S \to Aa|b$ $\quad A \to Bb|c$ $\quad B \to Cc|d$ $\quad C \to Sd|a$

10. Given grammar $G[S]$, and the production rules are as follows. Eliminate its left recursion.

    $S \to Aa|b$ $\quad A \to Bc|d$ $\quad B \to Se|Af|h$

11. Given grammar $G[S]$, and the production rules are as follows. Eliminate its left recursion.

    $S \to Aa$ $\quad S \to b$ $\quad A \to cS$ $\quad A \to Sd$

12. Given C-style if-statement grammar $G[S] : S \to if\ (B)\ S \mid if\ (B)\ S\ else\ S$. Extract the left common factor.
13. Given C-style while-statement grammar $G[S] : S \to which\ (B)\ S \mid which\ (B)\ \{S\}$, extracts the left common factor.
14. Given Matlab-style for-statement grammar $G[S] : S \to for\ E : E\ do\ S \mid for\ E : E : E\ do\ S$, extracts the left common factor.
15. The production rules of grammar $G[S]$ are as follows.

    $S \to ABCDE$ $\quad A \to a|\varepsilon$ $\quad B \to Bb|\varepsilon$
    $C \to Cc|\varepsilon$ $\quad D \to DE|\varepsilon$ $\quad E \to eS|\varepsilon$

    (1) Construct the $First$ set of all non terminal symbols.
    (2) Construct the $Follow$ set of all non terminal symbols.
16. The production rules of grammar $G[S]$ are as follows.

    $S \to ABCDE$ $\quad A \to Aa|\varepsilon$ $\quad B \to Bb|\varepsilon$
    $C \to DE$ $\quad D \to DE|\varepsilon$ $\quad E \to eS|\varepsilon$

    (1) Construct the $First$ set of all non terminal symbols.
    (2) Construct the $Follow$ set of all non terminal symbols.
17. The production rules of grammar $G[S]$ are as follows.

    $S \to ABCDE$ $\quad A \to BCDE$ $\quad B \to CDE$
    $C \to DE$ $\quad D \to DE|\varepsilon$ $\quad E \to eS|\varepsilon$

Yanwei Zheng, Shandong University

(1) Construct the $First$ set of all non terminal symbols.
(2) Construct the $Follow$ set of all non terminal symbols.

18. The production rules of grammar $G[S]$ are as follows.

$$S \to aBc \qquad S \to bAB \qquad A \to aAb$$
$$A \to b \qquad B \to b \qquad B \to \varepsilon$$

(1) Construct the $First$ set of all non terminal symbols.
(2) Construct the $Follow$ set of all non terminal symbols.
(3) Construct LL(1) analysis table.
(4) Write the analysis process of $baabbb$.
(5) Write the recursive descent analysis program.

19. The C-style declaration statement grammar $G[L]$ has the following production rules, where id is the variable name.

$$L \to L; D \qquad L \to D \qquad D \to T\ id$$
$$D \to D, id \qquad T \to int \qquad T \to double$$

(1) Eliminate its left recursion.
(2) Construct the $First$ set of all non terminal symbols.
(3) Construct the $Follow$ set of all non terminal symbols.
(4) Construct LL(1) analysis table.
(5) Write the analysis process of $int\ i; double\ x, y$.
(6) Write the recursive descent analysis program.

20. The Pascal-style declaration statement grammar $G[L]$ has the following production rules, where id is the variable name.

$$L \to L; D \qquad L \to D \qquad D \to id : T$$
$$D \to id, D \qquad T \to int \qquad T \to real$$

(1) Eliminate left recursion and left common factor in grammar.
(2) Construct the $First$ set of all non terminal symbols.
(3) Construct the $Follow$ set of all non terminal symbols.
(4) Construct LL(1) analysis table.
(5) Write the analysis process of $i : int; x, y : real$.
(6) Write the recursive descent analysis program.

21. The array use grammar $G[E]$ has the following production rules.

$$E \to E + T \qquad E \to T \qquad T \to T * F \qquad T \to F$$
$$F \to id \qquad F \to num \qquad F \to Elist] \qquad F \to (E)$$
$$Elist \to id[E \qquad Elist \to Elist, E$$

(1) Eliminate left recursion and left common factor in grammar.
(2) Construct the $First$ set of all non terminal symbols.
(3) Construct the $Follow$ set of all non terminal symbols.
(4) Construct LL(1) analysis table.
(5) Write the analysis process of $a[i, j] = a[i, j - 1] * 2$.
(6) Write the recursive descent analysis program.

Yanwei Zheng, Shandong University

22. The regular expression grammar $G[E]$ has the following production rules, where "|" is the OR operation in the regular expression, not the metalanguage of the grammar.

$$E \to E|T \qquad E \to T \qquad T \to TP \qquad T \to P$$
$$P \to P* \qquad P \to F \qquad F \to (E) \qquad F \to i$$

(1) Eliminate left recursion and left common factor in grammar.
(2) Construct the $First$ set of all non terminal symbols.
(3) Construct the $Follow$ set of all non terminal symbols.
(4) Construct LL(1) analysis table.
(5) Write the analysis process of $(a|b) * aa$.
(6) Write the recursive descent analysis program.

23. The simplified if-statement grammar $G[S]$ has the following production rules.

$$S \to iCtSE \qquad S \to s \qquad E \to eS$$
$$E \to \varepsilon \qquad C \to c$$

(1) Construct the $First$ set of all non terminal symbols.
(2) Construct the $Follow$ set of all non terminal symbols.
(3) Construct the LL(1) analysis table and intervene according to the following rules if a conflict occurs: If one alternative is a prefix of another alternative, the longer alternative takes precedence.
(4) Write the analysis process of $ictictses$.
(5) Write the recursive descent analysis program.

24. Matlab for $G[S]$ i -1 $-$ 1 The Matlab-style for-statement grammar $G[S]$ has the following production rules, where i represents any variable or constant (assuming that -1 is recognized as a constant during lexical analysis, rather than operators $-$ and constants 1).

$$S \to for\ E : E\ do\ S \qquad S \to for\ E : E : E\ do\ S \qquad S \to s$$
$$E \to E + T \qquad E \to T \qquad T \to i$$

(1) Eliminate left recursion and left common factor in grammar.
(2) Construct the $First$ set of all non terminal symbols.
(3) Construct the $Follow$ set of all non terminal symbols.
(4) Construct LL(1) analysis table.
(5) Write the analysis process of $for\ 100 : -1 : a + b\ do\ for\ 1 : 10\ do\ s$.
(6) Write the recursive descent analysis program.

25. The procedure call grammar $G[E]$ has the following production rules, where i represents any procedure name, variable name, or constant.

$$E \to id(Elist \qquad E \to id() \qquad Elist \to E, Elist \qquad Elist \to E)$$
$$E \to E + T \qquad E \to T \qquad T \to id$$

(1) Eliminate left recursion and left common factor in grammar.
(2) Construct the $First$ set of all non terminal symbols.
(3) Construct the $Follow$ set of all non terminal symbols.
(4) Construct LL(1) analysis table.
(5) Write the analysis process of $max(x, y + z)$.
(6) Write the recursive descent analysis program.

Yanwei Zheng, Shandong University

26. What are the priority relationships between operators?
27. Is the operator priority relationship a partial order relationship? Why?
28. What is operator grammar?
29. What is operator precedence grammar?
30. What is a prime phrase?
31. Briefly describe the leftmost prime phrase theorem.
32. The production rule of grammar $G[S]$ is as follows. Try to determine whether the grammar is an operator grammar. If not, transform it into an operator grammar.

    $S \to AB$ $\quad$ $A \to Aa|b$ $\quad$ $B \to aB|b$

33. The production rule of grammar $G[S]$ is as follows. Try to determine whether the grammar is an operator grammar. If not, transform it into an operator grammar.

    $S \to ABC$ $\quad$ $A \to Ao|s$ $\quad$ $B \to Bo|\varepsilon$ $\quad$ $C \to oC|s$

34. The production rule of grammar $G[S]$ is as follows. Try to determine whether the grammar is an operator grammar. If not, transform it into an operator grammar.

    $S \to ABC$ $\quad$ $A \to Ao|s$ $\quad$ $B \to B0|\varepsilon$ $\quad$ $C \to oC|s$

35. The production equation of grammar $G[S]$is as follows. Try to determine the priority relationship of the following operators: θ_1 and θ_1, θ_1 and θ_2, θ_1 and θ_3, θ_2 and θ_3, θ_3 and θ_3, θ_3 and θ_2, a and θ_1, a and θ_2, a and θ_3, θ_1 and a, θ_2 and a, θ_3 and a, a and a.

    $S \to S\theta_1 A$ $\quad$ $S \to A$ $\quad$ $A \to A\theta_2 B$ $\quad$ $A \to B$

    $B \to C\theta_3 B$ $\quad$ $B \to C$ $\quad$ $C \to (S)$ $\quad$ $C \to a$

36. Write the definitions of $FirstV_T(A)$ and $LastV_T(A)$ in the operator precedence analysis method, where A is any non terminal symbol.
37. What is operator precedence function?
38. Do all operator precedence tables have operator precedence functions? Why?
39. Is the operator precedence function in the operator precedence table unique? Why?
40. What are the advantages and disadvantages of precedence functions compared to precedence relationship tables?
41. The production rules of grammar $G[E]$ are as follows, where i represents any variable or constant.

    $E \to E + T$ $\quad$ $E \to T$ $\quad$ $T \to T * F$ $\quad$ $T \to F$

    $F \to (E)$ $\quad$ $F \to i$

    (1) Construct the $FirstV_T$ set of all non terminal symbols.
    (2) Construct the $LastV_T$ set of all non terminal symbols.
    (3) Calculate the operator precedence relations of terminal symbols.
    (4) Calculate the operator precedence functions of terminal symbols.
    (5) Write the analysis process of $(a + b) * 2$.

42. The production rules of grammar $G[S]$ are as follows.

    $S \to a$ $\quad$ $S \to \hat{}$ $\quad$ $S \to (T)$ $\quad$ $T \to T, S$

    $T \to S$

    (1) Construct the $FirstV_T$ set of all non terminal symbols.
    (2) Construct the $LastV_T$ set of all non terminal symbols.

Yanwei Zheng, Shandong University

(3) Calculate the operator precedence relations of terminal symbols.
(4) Calculate the operator precedence functions of terminal symbols.
(5) Write the analysis process of $(a, \hat{}, (a, a))$.

43. The production rules of grammar $G[E]$ are as follows, where i represents any variable or constant.

$$E \to (A, E, E) \qquad E \to i \qquad A \to + \qquad A \to *$$

(1) Construct the $FirstV_T$ set of all non terminal symbols.
(2) Construct the $LastV_T$ set of all non terminal symbols.
(3) Calculate the operator precedence relations of terminal symbols.
(4) Calculate the operator precedence functions of terminal symbols.
(5) Write the analysis process of $(*, (+, a, b), (+, a, b))$.

44. What are the meanings of each letter and number in LR(k) analysis method?
45. What methods does the LR analysis method family include? Briefly describe the characteristics of these methods.
46. What are the characteristics of LR analysis method?
47. Briefly describe the idea of LR(0) analysis.
48. What is a canonical sentential form? What is a viable prefix?
49. What is LR(k) item? What is an LR(1) valid item?
50. What is the difference between the SLR(1) analysis method and the LR(1) analysis method when viewing a symbol to the right?
51. Given a grammar $G[S] = (\{S, A\}, \{0, 1\}, P, S)$, which production rules are as follows.

$$S \to AS \qquad S \to 1 \qquad A \to SA \qquad A \to 0$$

(1) Augment the grammar.
(2) Construct canonical collection of LR(0) items for the augmented grammar.
(3) Calculate the $Follow$ set of the nonterminals for the augmented grammar.
(4) Construct the SLR(1) analysis table for the augmented grammar.

52. Prove that the grammar $G[S]$ below is an SLR(1) grammar, but not an LR(0) grammar.

$$S \to A \qquad A \to Ab \qquad A \to bBa$$
$$B \to aAc \qquad B \to aAb \qquad B \to a$$

53. Prove that the grammar $G[S]$ below is an LR(1) grammar, but not an SLR(1) grammar.

$$S \to A0A1 \qquad S \to B1B0 \qquad A \to \varepsilon \qquad B \to \varepsilon$$

54. The C-style declaration statement grammar $G[L]$ has the following production rules, where id is the variable name.

$$L \to L; D \qquad L \to D \qquad D \to T\ id$$
$$D \to D, id \qquad T \to int \qquad T \to double$$

(1) Augment the grammar.
(2) Construct canonical collection of LR(1) items for the augmented grammar.
(3) Construct LR(1) analysis table for the augmented grammar.
(4) Write the analysis process of $int\ a,\ b;\ double\ c$.

Yanwei Zheng, Shandong University

55. The Pascal-style declaration statement grammar $G[L]$ has the following production rules, where id is the variable name.

$$L \to L; D \qquad L \to D \qquad D \to id : T$$
$$D \to id, D \qquad T \to int \qquad T \to real$$

(1) Augment the grammar.
(2) Construct canonical collection of LR(1) items for the augmented grammar.
(3) Construct LR(1) analysis table for the augmented grammar.
(4) Write the analysis process of $a,\ b:\ int;\ c:\ real$.

56. The procedure call grammar $G[E]$ has the following production rules, where id is the process name, variable name, or constant.

$$E \to id(L \qquad L \to E, L \qquad L \to E) \qquad E \to id$$

(1) Augment the grammar.
(2) Construct canonical collection of LR(1) items for the augmented grammar.
(3) Construct LR(1) analysis table for the augmented grammar.
(4) Write the analysis process of $s(x, y)$.

57. The regular expression grammar $G[E]$ has the following production rules, where i is any character.

$$E \to E|E \qquad E \to EE \qquad E \to E*$$
$$E \to (E) \qquad E \to i$$

(1) Augment the grammar.
(2) Construct canonical collection of LR(1) items for the augmented grammar.
(3) Construct LR(1) analysis table for the augmented grammar. If there is a conflict, intervene according to the following rules: NOT, AND, OR priority from high to low, NOT operator is combined on the right, and AND, OR operators are combined on the left.
(4) Write the analysis process of $(a|b) * a$.

58. The if-statement grammar $G[S]$ has the following production rules.

$$S \to iCtSE \qquad S \to s \qquad E \to eS$$
$$E \to \varepsilon \qquad C \to c$$

(1) Augment the grammar.
(2) Construct canonical collection of LR(1) items for the augmented grammar.
(3) Construct LR(1) analysis table for the augmented grammar. If there is a conflict, intervene according to the following rule: If one candidate is a prefix of another, the longer one takes precedence.
(4) Write the analysis process of $ictictses$.

59. Matlab for $G[S]$ i -1 $-$ 1 The Matlab-style for-statement grammar $G[S]$ has the following production rules, where i represents any variable or constant (assuming that -1 is identified as a constant during lexical analysis, rather than operator $-$ and constant 1).

$$S \to for\ E : E\ do\ S \qquad S \to for\ E : E : E\ do\ S \qquad S \to s$$
$$E \to E + T \qquad E \to T \qquad T \to i$$

(1) Augment the grammar.

Yanwei Zheng, Shandong University

(2) Construct canonical collection of LR(1) items for the augmented grammar.
(3) Construct LR(1) analysis table for the augmented grammar.
(4) Write the analysis process of $for\ 100 : -1 : a + b\ do\ for\ 1 : 10\ do\ s$.

60. What is congeneric sets?
61. Will merging concentric sets generate new transition-reduction conflicts? Why?
62. The grammar $G[E]$ has the following rules.

$$E \to E * E \qquad E \to E \hat{} E \qquad E \to (E) \qquad E \to i$$

(1) Augment the grammar.
(2) Construct canonical collection of LR(1) items for the augmented grammar.
(3) If possible, merge concentric sets.
(4) Construct LALR(1) analysis table for the augmented grammar. If there is a conflict, intervene according to the following rules: power operation takes priority over multiplication operation, multiplication is combined on the left, and power operation is combined on the right.
(5) Write the analysis process of $i * i \hat{} i \hat{} i$.

63. The grammar $G[E]$ for a system consisting of negative operations and subtraction is as follows.

$$E \to E - E \qquad E \to -E \qquad E \to (E) \qquad E \to i$$

(1) Augment the grammar.
(2) Construct canonical collection of LR(1) items for the augmented grammar.
(3) If possible, merge concentric sets.
(4) Construct LALR(1) analysis table for the augmented grammar. If there is a conflict, intervene according to the following rules: negative operations have higher priority than subtraction operations, subtraction is left combined, and negative operations are right combined;
(5) Write the analysis process of $-(i - - - i)$.

64. The grammar $G[S]$ is as follows.

$$S \to AA \qquad A \to oA \qquad A \to s$$

(1) Augment the grammar.
(2) Construct canonical collection of LR(1) items for the augmented grammar.
(3) If possible, merge concentric sets.
(4) Construct LALR(1) analysis table.
(5) Write the analysis process of sos.

65. The grammar $G[S]$ is as follows.

$$S \to xXu \qquad S \to yYu \qquad S \to yXv \qquad S \to xYv$$
$$X \to w \qquad Y \to w$$

(1) Augment the grammar.
(2) Construct canonical collection of LR(1) items for the augmented grammar.
(3) Find the concentric set
(4) Can concentric sets be merged? Why?

66. The sentence sequence grammar $G[S]$ has the following production rules, where s represents any sentence, and M is the null production rule introduced for inserting actions during sentence translation.

Yanwei Zheng, Shandong University

$S \to \{L\}$	$L \to S$	$L \to LMS$
$S \to s;$	$M \to \varepsilon$	

(1) Augment the grammar.
(2) Construct canonical collection of LR(1) items for the augmented grammar.
(3) Construct LR(1) analysis table.
(4) Write the analysis process of $\{sss\}$.

67. Pascal-style procedural declaration statement grammar $G[S]$, the production rules are as follows, where id represents the variable name, semicolons are left combined, and M is the null production rule introduced during statement translation for inserting actions.

$P \to MD$	$M \to \varepsilon$	$D \to D; D$
$D \to id\ L$	$L \to, id\ L$	$L \to: T$
$T \to int$	$T \to real$	

(1) Augment the grammar.
(2) Construct canonical collection of LR(1) items for the augmented grammar.
(3) Construct LR(1) analysis table.
(4) Write the analysis process of $i, j : int; a : real$.

68. Pascal-style array declaration statement grammar $G[V]$, the production rules are as follows, where id represents the variable name, num is the constant, and arr is the the array name.

$V \to id\ L$	$L \to, id\ L$	$L \to: T$
$T \to int$	$T \to real$	$A \to arr[num : num$
$A \to A, num : num$	$T \to A]\ of\ T$	

(1) Augment the grammar.
(2) Construct canonical collection of LR(1) items for the augmented grammar.
(3) Construct LR(1) analysis table.
(4) Write the analysis process of $a : arr\ [0 : 2,\ 3 : 7]\ of\ int$.

69. The C-style procedure definition statement grammar $G[F]$ has the following production rules, where id represents the procedure name or variable name, N is the null production rule introduced for inserting actions during statement translation, and s is any program statement other than the procedure definition.

$F \to HM)NS$	$H \to T\ id($	$N \to \varepsilon$
$M \to V$	$M \to M, V$	$V \to T\ id$
$S \to s$	$T \to void$	$T \to int$

(1) Augment the grammar.
(2) Construct canonical collection of LR(1) items for the augmented grammar.
(3) Construct LR(1) analysis table.
(4) Write the analysis process of $int\ sum(int\ x,\ int\ y)$.

70. The C-style struct definition statement grammar $G[S]$ has the following production rules, where id represents the struct name or variable name.

$S \to RM\};$	$S \to RM\}\ id;$	$R \to struct\ id\{$
$M \to V$	$M \to MV$	$V \to T\ id;$
$T \to int$	$T \to real$	$T \to struct\ id$

Yanwei Zheng, Shandong University

(1) Augment the grammar.
(2) Construct canonical collection of LR(1) items for the augmented grammar.
(3) Construct LR(1) analysis table.
(4) Write the analysis process of $struct\ PT\{int\ x,\ int\ y;\ struct\ RGB\ color;\};$.

71. The control statement grammar $G[S]$ has the following production rules, where s represents any statement, b is a Boolean expression, and M and N are null production rules introduced for inserting actions during statement translation.

$S \to if\ (B)\ MSN\ else\ MS$ $\quad S \to if\ (B)\ MS$ $\quad S \to while\ (MB)\ MS$
$S \to s$ $\quad B \to b$ $\quad M \to \varepsilon$
$N \to \varepsilon$

(1) Augment the grammar.
(2) Construct canonical collection of LR(1) items for the augmented grammar.
(3) Construct LR(1) analysis table.
(4) Write the analysis process of $while\ (b)\ if\ (b)\ s\ else\ s$.

72. Matlab-style for-statement grammar $G[S]$, the production rules are as follows, where s represents any statement, e is an arithmetic expression, M is a null production rule introduced for inserting actions during statement translation, and id is the variable name.

$S \to for(H)S$ $\quad H \to JME$ $\quad H \to JME : E$
$J \to id = E :$ $\quad S \to s$ $\quad M \to \varepsilon$

(1) Augment the grammar.
(2) Construct canonical collection of LR(1) items for the augmented grammar.
(3) Construct LR(1) analysis table.
(4) Write the analysis process of $for\ (i = 100 : -1 : 0)\ for\ (j = 1 : 10)\ s$ (Assuming that -1 is recognized as a constant during lexical analysis, instead of the operator $-$ and the constant 1).

73. Label-goto statement grammar $G[S]$, the production rules are as follows, where s represents any statement and id represents the label name.

$S \to label\ S$ $\quad label \to id :$ $\quad S \to goto\ id;$
$S \to s$

(1) Augment the grammar.
(2) Construct canonical collection of LR(1) items for the augmented grammar.
(3) Construct LR(1) analysis table.
(4) Write the analysis process of $goto\ a;\ a:\ s\ goto\ a;$.

74. The Switch-statement grammar $G[S]$ has the following production rules, where s represents any statement, e represents an arithmetic expression, and c represents a constant.

$S \to W\{UX\}$ $\quad W \to switch(E)$ $\quad E \to e$
$U \to CS$ $\quad C \to case\ c :$ $\quad C \to U\ case\ c :$
$X \to default : MS$ $\quad X \to \varepsilon$ $\quad M \to \varepsilon$
$S \to s$

(1) Augment the grammar.
(2) Construct canonical collection of LR(1) items for the augmented grammar.
(3) Construct LR(1) analysis table.

Yanwei Zheng, Shandong University

(4) Write the analysis process of $switch(e)\ \{case\ 0:\ s\ case\ 1:\ s\ default:\ s\}$.

75. The Lisp-style expression grammar $G[E]$ has the following production rules, where i represents any variable or constant.

$$E \to (A, E, E) \qquad E \to i \qquad A \to + \qquad A \to *$$

(1) Augment the grammar.

(2) Construct canonical collection of LR(1) items for the augmented grammar.

(3) Construct LR(1) analysis table.

(4) Write the analysis process of $(*, (+, a, b), (+, a, b))$.

Yanwei Zheng, Shandong University

Chapter 5 Symbol Table

Chapter 5 Exercise

1. What is Symbol Table?
2. What is the function of a symbol table?
3. What are the operations on the symbol table during compilation?
4. List the variable items of symbol table.
5. List the array items of symbol table.
6. List the structure items of symbol table.
7. List the procedure items of symbol table.
8. List the label items of symbol table.
9. Explain the variable scope rules of Pascal-style nested definition procedure.
10. Explain the symbol table structure organization of Pascal-style nested definition procedure.
11. Explain the working process of the symbol table stack in Pascal-style nested definition procedure.
12. What are the ways to organize the content of symbol tables?
13. Explain the table organization of the symbol table.
14. Explain the organization of symbol table records.
15. Explain the object-oriented organization of symbol tables.
16. Explain the big-endian and small-endian encoding of integers, and write the two 4-byte encoding forms for 0AB01h.
17. What are the storage methods for strings? Write the byte sequence of the string "abcd" using different storage methods.
18. Write the symbol table for the following code.

```
struct STRU {
  int a;
  float b[5, 10];
  double c;
};
int Fun(STRU s, int len){
  STRU temp;
  int i, j;
}
```

19. Write the table records of the variables in the table below to store their contents.

Name	Data Type	Category	Width	Offset
x	int	Formal	4	0
y	int	Formal	4	4
i	int	Variable	4	0
j	int	Variable	4	4
u	double	Variable	8	8
v	double	Variable	8	16

Lanwei Zheng, Shandong University

arr	array(2, 3, 4, int)	Variable	96	24

20. Write the linear list of variables x, y, i, j, u, v, and arr.
21. Compare the sizes of variables x, y, i, j, u, v, and arr according to their name strings, and write the binary tree of the variables.
22. Compare the sizes of variables x, y, i, j, u, v, and arr according to their name strings, and write the balanced binary tree of the variables.
23. For variables x, y, i, j, u, v, arr, add up their ASCII code values according to their names and convert them into numerical representations. Construct a storage mapping function using the direct addressing method, perform linear probing and hash to resolve conflicts, and write the hash table for the variables.
24. For variables x, y, i, j, u, v, arr, add up their ASCII code values according to their names and convert them into numerical representations. Construct a storage mapping function using the direct addressing method, perform square detection and hash to resolve conflicts, and write the hash table for the variables.
25. For variables x, y, i, j, u, v, arr, add up their ASCII code values according to their names and convert them into numerical representations. Construct a storage mapping function using the direct addressing method, resolve conflicts using the chain address method, and write the hash table of the variables.
26. Write the linear list of variables prod, factorial, fact, num1, num2, var, and visible.
27. Compare the sizes of variables prod, factorial, fact, num1, num2, var, and visible by name string, and write the binary tree of the variables.
28. Compare the sizes of variables prod, factorial, fact, num1, num2, var, and visible by name string, and write the balanced binary tree of the variables.
29. For variables prod, factorial, fact, num1, num2, var, and visible, add up the ASCII code values according to their names and convert them into numerical representations. Construct a storage mapping function using direct addressing method, perform linear probing and hash to resolve conflicts, and write the hash table of the variables.
30. For variables prod, factorial, fact, num1, num2, var, and visible, add up the ASCII code values according to their names and convert them into numerical representations. Construct a storage mapping function using direct addressing method, perform square detection and hash to resolve conflicts, and write the hash table of the variables.
31. For variables prod, factorial, fact, num1, num2, var, and visible, add up the ASCII code values according to their names and convert them into numerical representations. Construct a storage mapping function using the direct addressing method, resolve conflicts using the chain address method, and write the hash table of the variables.

Yanwei Zheng, Shandong University

Chapter 6 Runtime Storage Organization

Chapter 6 Exercise

1. Describe the access methods for different storage segments during runtime.
2. What is activity record? Is the activity record necessarily a stack frame?
3. Draw an activity record of the procedure and explain the contents stored in each part.
4. What are the runtime storage allocation strategies? When to use what storage strategy?
5. What are the methods of argument passing in high-level programming languages? How was it done specifically?
6. What are the procedure call specifications?
7. Briefly describe the std call.
8. Briefly describe the C Declaration.
9. Briefly describe the x64 procedure call specification.
10. Briefly describe the ARM procedure call specification.
11. The factorial() procedure passes an integer parameter n and calculates n! Recursively. The main() procedure calls factorial() and uses the std call specification. Write the assembly code.
12. The factorial() procedure passes an integer parameter n and calculates n! Recursively. The main() procedure calls factorial() and uses the C Declaration specification. Write the assembly code.
13. The factorial() procedure passes an integer parameter n and calculates n! Recursively. The main() procedure calls factorial() and uses the x64 procedure call specification. Write the assembly code.
14. The swap() procedure accepts two address parameters, and after execution, exchanges the contents stored in the two addresses. The main() process uses the push instruction to push in the numbers 0 and 1, and then passes the addresses of these two numbers to the swap() procedure for calling. Write assembly code using the std call specification.
15. The swap() procedure accepts two address parameters, and after execution, exchanges the contents stored in the two addresses. The main() process uses the push instruction to push in the numbers 0 and 1, and then passes the addresses of these two numbers to the swap() procedure for calling. Write assembly code using the C Declaration specification.
16. The swap() procedure accepts two address parameters, and after execution, exchanges the contents stored in the two addresses. The main() process uses the push instruction to push in the numbers 0 and 1, and then passes the addresses of these two numbers to the swap() procedure for calling. Write assembly code using the x64 procedure call specification.
17. The factorial() procedure passes an integer parameter n and recursively calls to calculate n!. The main() procedure calls factorial(3), using the std call specification. Draw figures to represent the activity record for each process call.
18. The factorial() procedure passes an integer parameter n and recursively calls to calculate n!. The main() procedure calls factorial(3), using the C Declaration specification. Draw figures to represent the activity record for each process call.
19. The factorial() procedure passes an integer parameter n and recursively calls to calculate n!. The main() procedure calls factorial(3), using the x64 procedure call specification. Draw figures

Yanwei Zheng, Shandong University

to represent the activity record for each process call.

20. Using the structural object as a procedure parameter and passing values for parameter transmission, design the strategy.

21. Write the stack frame after entering the function Fun according to the following code.

```
struct STRU {
  int a;
  float b[5, 10];
  double c;
};
int Fun(STRU s, int len){
  STRU temp;
  int i, j;
}
int main() {
  STRU a;
  ...
  Fun(a, 10);
  ...
  return 0;
}
```

22. Write the access address for accessing the following variables through the EBP register according to the following code: len, s.a, s.b[2, 5], s.c, temp.a, temp.b[2, 5], temp.c, i, and j.

```
struct STRU {
  int a;
  float b[5, 10];
  double c;
};
int Fun(STRU s, int len){
  STRU temp;
  int i, j;
}
int main() {
  STRU a;
  ...
  Fun(a, 10);
  ...
  return 0;
}
```

23. Write the stack frame after entering the function Fun according to the following code.

```
struct STRU {
  int a;
  float b[5, 10];
  double c;
};
int Fun(STRU &s, int len){
```

Yanwei Zheng, Shandong University

```
    STRU temp;
    int i, j;
  }
  int main() {
    STRU a;
     ...
    Fun(a, 10);
     ...
    return 0;
  }
```

24. Write the access address for the following variables through the EBP according to the following code: len, s.a, s.b[2, 5], s.c, temp.a, temp.b[2, 5], temp.c, i, and j.

```
  struct STRU {
    int a;
    float b[5, 10];
    double c;
  };
  int Fun(STRU &s, int len){
    STRU temp;
    int i, j;
  }
  int main() {
    STRU a;
     ...
    Fun(a, 10);
     ...
    return 0;
  }
```

Yanwei Zheng, Shandong University

25. Design a console program using x86 assembly, which calls the printf function of the C runtime library to print the contents of all general-purpose registers to the screen.
26. Describe the methods of cross file calling.
27. Describe the methods of encapsulating libraries and calling libraries.
28. Describe the construction method of static chains.
29. Describe the method of accessing outer variables through static chains.
30. What is a nested hierarchical display table?
31. Describe the method of accessing variables through the Display table.
32. According to the following Pascal codes, what is the content of Display table when recursively entering the factorial function for the second time? What is the content of the stack frame at this time?

```
  program Pgrm(input, output)
    var result : integer;
    function factorial (n : integer) : integer
    begin
      if n <= 0 then factorial := 1
      else factorial := n * factorial (n - 1);
```

```
  end;
begin
  result = factorial(10);
  ...
end.
```

33. What are the ways to allocate heap storage? How was it done specifically?
34. What are the ways to recycle heap storage? How was it done specifically?
35. The following C++ code are used to allocate and release heap memory by calling the malloc() and free() functions of the C runtime library. Implementing it using assembly code.

```
int *p = new int[20];
...
delete p;
```

Yanwei Zheng, Shandong University

Chapter 7 Grammar Guided Translation and Intermediate Code Generation

Chapter 7 Exercise

1. What are the tasks of static semantic checking?
2. Describe the attributes of quadruple.
3. What is attribute grammar?
4. What is synthetic attribute? What is inheritance property?
5. What types of attributes can a terminator have? How to calculate?
6. How is the inheritance attribute of the start symbol calculated?
7. What is grammar guided translation?
8. What is S-attribute grammar? What is L-attribute grammar?
9. What is a dependency graph for attribute computation? How is it constructed?
10. The following grammar $G[E]$ applies addition and multiplication operations to integer and real constants. Add two integers to obtain an integer result, otherwise the result is real. Provide a attribute grammar that determines the data type of each production rule.

    $E \to E + T$ $\quad E \to T$ $\quad T \to T * F$ $\quad T \to F$

    $F \to (E)$ $\quad F \to num.num$ $\quad F \to num$

11. The following grammar $G[S]$ generates a binary number. Design a synthetic attribute val, where $S.val$ is the decimal value corresponding to a binary number recognized. For example, when entering 101.01, $S.val = 5.25$.

    $S \to L.R$ $\quad S \to L$ $\quad L \to LB$

    $L \to B$ $\quad R \to RB$ $\quad R \to B$

    $B \to 0$ $\quad B \to 1$

12. Construct an abstract syntax tree for sentence $(a+b)*(a+b)$ based on the S-attribute grammar used to construct the abstract syntax tree.
13. According to the S-attribute grammar of constructing single- or ε- symbol NFA, construct an NFA for sentence $(a|b)^*(aa|bb)$.
14. Modify the attribute grammar of the desktop calculator to eliminate left recursion and enable top-down calculations.

Serial	Production Rules	Semantic Rules
1	$L \to E$	$print(E.value)$;
2	$E \to E_1 + E_2$	$E.value = E_1.value + E_2.value$;
3	$E \to E_1 * E_2$	$E.value = E_1.value * E_2.value$;
4	$E \to (E_1)$	$E.value = E_1.value$;
5	$E \to num$	$E.value = num.lexvalue$;

15. Translate Pascal-style declaration statements “a, b: integer; x, y : real”.
16. Translate Pascal-style declaration statements “arr1, arr2 : array[1..5, 2..7] of real; a, b : integer”.
17. Translate C-style declaration statements “int a, b; double x, y;”.
18. Translate C-style declaration statements “int a[2, 4]; double x, y;”.

Yanwei Zheng Shandong University

19. Translate the following C-style declaration statements.

```
int sum(int x, y;) {
  int sum;
}
```

20. Translate the following C-style declaration statements.

```
struct STRU {
  int a;
  float b[5, 10];
  double c;
};
int Fun(STRU s; int u, v;) {
  STRU temp;
  int i, j;
}
```

21. Translate the following C-style expression assignment statement "delt = b * b - 4 * a * c;".
22. Translate the following C-style expression assignment statement "a[i, j] = a[i, 2 * j] * 2;".
23. Translate the following C-style expression assignment statement "temp.b[i, j] = s.b[i, j];", where s and temp are the formal parameters and local variables of the Fun function in the following code.

```
struct STRU {
  int a;
  float b[5, 10];
  double c;
};
int Fun(STRU s; int u, v;) {
  STRU temp;
  int i, j;
}
```

24. Translate the following C-style expression assignment statement "isMin = x <= y && x <= z;".
25. Translate the following C-style expression assignment statement "p = &x; y = *p + x;".
26. The production rules for self increasing and self decreasing expressions in C language are as follows, where *id* is the variable name. Design translation schemes based on the semantic rules of C language.

$$E \to id++ \qquad E \to ++id \qquad E \to id-- \qquad E \to --id$$

27. The production rules of the composite assignment operator in C language are as follows, where *id* is the variable name and *num* is the constant. Design translation schemes based on the semantic rules of C language.

$$E \to id+=E \qquad E \to id*=E \qquad E \to E+E \qquad E \to E*E$$
$$E \to (E) \qquad E \to id \qquad E \to num$$

28. Design translation schemes for self increasing and self decreasing expressions, combined with expressions and assignment statements, to translate the following sentence "a = (++b) + (b−−);".

Yanwei Zheng, Shandong University

29. Design translation schemes for composite assignment operations, combining expressions and assignment statements, to translate the following sentence "a += b * c;".
30. Why do different compilers produce different results for expressions containing the same variable that self increment/decrement, such as "i+++−−i"?
31. Translate sentence of "if (x < i + j && i > j) i = i − 1; x = i; ".
32. Translate sentence of "if (x < i + j && i > j) i = i − 1; else j = j − 1; x = i;".
33. Translate sentence of "while (x < i + j && i > j) i = i − 1; x = i; ".
34. Translate sentence of "while (x < i + j && i > j) if (j > 10) i = i − 1; x = i;".
35. Translate sentence of "while (x < i + j && i > j) if (j > 10) i = i − 1; else i = i + 1; x = i;".
36. Translate sentence of "if (x < i + j && i > j) while (i > 0) i = i − 1; x = i;".
37. Translate sentence of "while (x < i + j) while (i > j) i = i − 1; x = i;".
38. Translate sentence of "for (i = 1; i < 10; i = i + 1;) sum = sum + i; i = 0;".
39. Translate sentence of "for (i = 1 : 10;) sum = sum + i; i = 0;".
40. Translate sentence of "for (i = 10 : −1 : 1;) sum = sum + i; i = 0;".
41. Translate sentence of "for (i = 1 : 2 : n;) {sum = sum + i; n = n + 1;} i = 0;".
42. Translate sentence of "for (i = 1 : 2 : n;) for (j = 1 : n) x = x + i + j; i = 0;".
43. Translate sentence of "if (i = j) goto L; if (i = 2 * j) goto L; L: x = y + z; goto L;".
44. Translate sentence of "switch(month){case 2 : day = 28; case 4 : day = 30; case 6: day = 30; case 9: day = 30; case 11: day = 30; default : day = 31; } day = day * 4;".
45. Translate sentence of "x = n % 2 == 0 ? y + z : y − z;", where "%", "=", and '==" are remainder, assignment, and relation operators, respectively.
46. Translate sentence of "while (a <= i <= b < j) {i = i + 1; a = a + 2;} i = a + b;".
47. Translate sentence of "for (i = 1 : 10;) {sum = sum + i; if (sum > 10) break; } i = 0;".
48. Translate sentence of "x = a > b && a < c ? a : c;".
49. Translate sentence of "if (a > b > c > d) x = a; y = d;".
50. Translate sentence of "if (a < b = c > d) x = a + b; y = c + d;".
51. Translate sentence of "z = a > b ? sum(a * 2, b) : sum(a, b * 2);".
52. What is type checking? What is a good type system?
53. Taking C language as an example, write type expressions.
54. Write the translation schemes for type checking of arithmetic operations in C language.
55. Write the translation schemes for type checking of relational operations in C language.
56. Write the translation schemes for type checking of logical operations in C language.
57. Write the translation schemes for type checking of reference operations in C language.
58. Write the translation schemes for type checking of structure using in C language.
59. Translate the sentence of "x=(y+i) * j;", where x and y are real variables, i and j are integer variables.
60. Translate the sentence of "i = (int)(x + y) * j;", where x and y are real variables, i and j are integer variables.
61. Translate the sentence of "i = (int)((x + y) * j);", where x and y are real variables, i and j are integer variables.
62. Translate the sentence of "x = (real)(i + j) * y;", where x and y are real variables, i and j are integer variables.
63. If the system only supports constant strings, printf("abc\r\n") can be translated into quadru-

Yanwei Zheng, Shandong University

ples $(param, \text{“}abc\backslash r\backslash n\text{”}, -, -)$ and $(call, printf, -, -)$. To support this operation, simply add a semantic rule of $E \rightarrow cstr\{E.val = cstr.val\}$, where $cstr$ represents a constant string and its attribute val comes from the lexical analyzer. When the lexical analyzer scans the constant string “abc\r\n”, $cstr.val =$“$abc\backslash r\backslash n$” (the lexical analyzer will replace escape characters with real characters). Based on this design, try translating the following statement: printf(“%d + %d = %d\r\n”, x, y, x + y);

64. Translate the following code.

```
int sum(int x, y;) {
  return x + y;
}
int main()
{
  int a, b;
  printf (“ Please input two numbers: \r\n”);
  scanf(“%d + %d”, &a, &b);
  printf (“  = %d\r\n”, sum(a, b));
  return 0;
}
```

65. Translate the following code.

```
int factorial (int n;) {
  if (n = 1)
    return 1;
  else
    return n * factorial (n - 1);
}
int main() {
  int n;
  printf (“ Please input a number: \r\n”);
  scanf(“%d”, &n);
  printf(“%d! = %d\r\n”, n, factorial (n));
  return 0;
}
```

Yanwei Zheng, Shandong University

66. Translate the following quick sort code, where the formal parameters need to modify the translation schemes of the declaration statement to accept arrays without specifying dimension information; Alternatively, you can modify the first parameter in the code to a pointer type and access the element through address offset.

```
void swap(int *x; int *y;) {
  int t = *x;
  *x = *y;
  *y = t;
}
void quickSort(int arr []; int start, end;) {
  if (start >= end)
    return;
  int pivot = arr[end];
```

```
    int left = start, right = end - 1;
    while (left < right) {
      while (arr[left] < pivot && left < right)
        left = left + 1;
      while (arr[right] >= pivot && left < right)
        right = right - 1;
      swap(&arr[left], &arr[right]);
    }
    if (arr[left] >= arr[end])
      swap(&arr[left], &arr[end]);
    else
      left = left + 1;
    if (left > 0)
      quickSort(arr, start, left - 1);
    quickSort(arr, left + 1, end);
  }
```

67. This chapter did not consider byte alignment when calculating variable offsets. If hardware reads a 32-bit double-word at once, storing a number across double-word will result in the reading twice to retrieve that number. For this, it is necessary to align the storage to a multiple of 4 bytes. If a byte type and an int type are stored consecutively, 3 bytes need to be left empty after the byte type; Two byte types and one int type are stored consecutively, two byte types are stored consecutively, followed by 2 empty bytes. The ultimate goal is to ensure that a complete data does not cross dual word storage. According to this agreement, design the calculation method for $offset$ in the declaration statement translation mode.
68. If you want to support reference calls for formal parameters, consider how to design the semantic rules for procedure definition and procedure calls, and how to design the symbol table.

Yanwei Zheng, Shandong University

Chapter 8

Chapter 8 Exercise

1. What are the basic block entry statements?
2. How to divide the basic blocks after determining the entry statement of the basic block?
3. What are the rules for constructing a control flow graph (CFG)?
4. What are dominating nodes and dominating node sets?
5. What are the properties of a loop? What is the definition of a loop?
6. What is back edge?
7. What is strict dominate? What is direct dominate?
8. What is a dominant tree?
9. For the following code.

$100.(proc, evenSum, -, -)$	$104.(j, -, -, 110)$	$108.(=, \$3, -, i)$
$101.(=, 1, -, i)$	$105.(+, sum, i, \$2)$	$109.(j, -, -, 103)$
$102.(=, 0, -, sum)$	$106.(=, \$2, -, sum)$	$110.(ret, sum, -, -)$
$103.(j \leq, i, n, 105)$	$107.(+, i, 1, \$3)$	$111.(endp, evenSum, -, -)$

   (1) Divide basic blocks.
   (2) Construct the control flow graph.

10. The code for summing up even numbers is as follows.

$100.(proc, evenSum, -, -)$	$105.(\%, i, 2, \$1)$	$110.(+, i, 1, \$3)$
$101.(=, 1, -, i)$	$106.(j =, \$1, 0, 108)$	$111.(=, \$3, -, i)$
$102.(=, 0, -, sum)$	$107.(j, -, -, 110)$	$112.(j, -, -, 103)$
$103.(j \leq, i, n, 105)$	$108.(+, sum, i, \$2)$	$113.(ret, sum, -, -)$
$104.(j, -, -, 113)$	$109.(=, \$2, -, sum)$	$114.(endp, evenSum, -, -)$

   (1) Divide basic blocks.
   (2) Construct the control flow graph.

11. For the following factorial code.

$100.(proc, fact, -, -)$	$104.(-, n, 1, \$1)$	$108.(*, n, \$2, \$3)$
$101.(j =, n, 1, 103)$	$105.(param, \$1, -, -)$	$109.(ret, \$3, -, -)$
$102.(j, -, -, 104)$	$106.(call, fact, -, -)$	$110.(endp, fact, -, -)$
$103.(ret, 1, -, -)$	$107.(=, fact, -, \$2)$	

   (1) Divide basic blocks.
   (2) Construct the control flow graph.

12. For the following code.

$100.(proc, fun, -, -)$	$109.(=, 1, -, j)$	$118.(+, sum1, j, \$4)$
$101.(=, 0, -, sum1)$	$110.(j \leq, j, i, 115)$	$119.(=, \$4, -, sum1)$
$102.(=, 0, -, sum2)$	$111.(j, -, -, 106)$	$120.(j, -, -, 112)$
$103.(=, 1, -, i)$	$112.(+, j, 1, \$2)$	$121.(+, sum2, j, \$5)$
$104.(j \leq, i, n, 109)$	$113.(=, \$2, -, j)$	$122.(=, \$5, -, sum2)$
$105.(j, -, -, 124)$	$114.(j, -, -, 110)$	$123.(j, -, -, 112)$

Yanwei Zheng, Shandong University

106.$(+, i, 1, \$1)$	115.$(\%, j, 2, \$3)$	124.$(*, sum1, sum2, \$6)$
107.$(=, \$1, -, i)$	116.$(j =, \$3, 0, 118)$	125.$(ret, \$6, -, -)$
108.$(j, -, -, 104)$	117.$(j, -, -, 121)$	126.$(endp, fun, -, -)$

(1) Divide basic blocks.
(2) Construct the control flow graph.
(3) Calculate the dominated node set of each basic block.
(4) Find the back edges.
(5) Find the loops.
(6) Calculate the levels of each loop.
(7) Construct the dominant tree.

13. Fig. 8.1 is a CFG containing some loops, where $B1$ and $B7$ are the entry and exit basic block respectively.

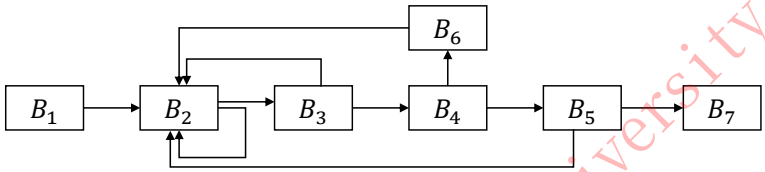


Figure 8.1 CFG of loops

(1) Calculate the dominated node set of each basic block.
(2) Find the back edges.
(3) Find the loops.
(4) Calculate the levels of each loop.
(5) Construct the dominant tree.

14. Fig. 8.2 is a CFG, where $B1$ and $B8$ are the entry and exit basic block respectively.

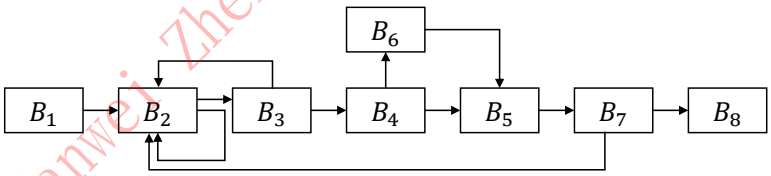


Figure 8.2 CFG of loops

(1) Calculate the dominated node set of each basic block.
(2) Find the back edges.
(3) Find the loops.
(4) Calculate the levels of each loop.
(5) Construct the dominant tree.

15. What are the basic principles of optimization?
16. What are definition and use of variable?
17. What is the live of a variable?
18. What is deleting common subexpressions?
19. What is copy propagation?
20. What is deleting useless assignments?
21. What is loop-invariant code motion (code hoisting)?

Yanwei Zheng, Shandong University

22. What is strength reduction?
23. What is constant folding?
24. What is induction variable elimination?
25. What is a DAG of basic blocks?
26. From the perspective of DAG, what are the types of quadruple? Draw its DAGs.
27. Briefly describe the ideas of DAG optimization.
28. Draw a framework figure of the DAG optimization algorithm.
29. Perform DAG optimization on the following code.

$100.(+, x, y, \$1)$	$103.(-, x, y, \$1)$	$106.(+, \$1, \$2, u)$
$101.(-, x, y, \$2)$	$104.(+, \$1, \$3, \$2)$	$107.(+, \$1, \$2, v)$
$102.(*, \$1, \$2, \$3)$	$105.(+, \$1, \$2, \$3)$	

30. Perform DAG optimization on the following code.

$100.(+, x, y, \$1)$	$103.(+, x, y, \$3)$	$106.(+, \$1, u, v)$
$101.(-, x, y, \$2)$	$104.(-, x, y, \$4)$	$107.(+, \$2, v, u)$
$102.(*, \$1, \$2, u)$	$105.(*, \$3, \$4, v)$	$108.(*, \$3, \$4, u)$

31. Perform DAG optimization on the following code.

$100.(+, 1, x, \$1)$	$103.(=, 1, -, x)$	$106.(+, \$3, 1, u)$
$101.(*, 2, x, \$2)$	$104.(+, x, y, \$3)$	$107.(+, x, 1, v)$
$102.(*, \$1, \$2, \$3)$	$105.(+, \$3, x, \$4)$	$108.(j <, x, v, 111)$

32. Perform DAG optimization on the following code.

$100.(+, 1, x, \$1)$	$103.(=, 1, -, x)$	$106.(+, \$3, \$4, u)$
$101.(*, 2, x, \$2)$	$104.(=, 2, -, y)$	$107.(+, x, 1, v)$
$102.(*, \$1, \$2, \$3)$	$105.(+, x, y, \$4)$	$108.(j <, u, v, 111)$

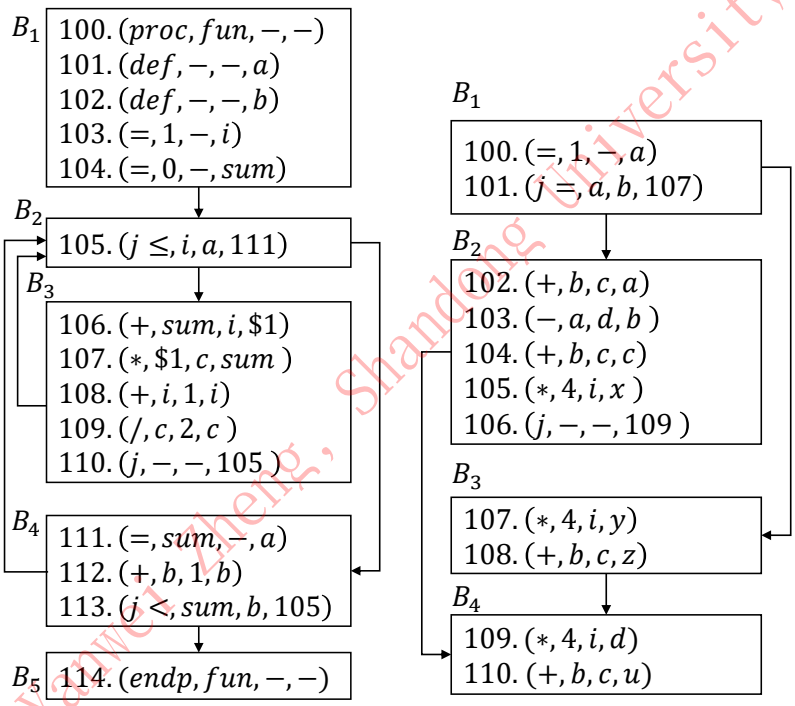
33. Perform DAG optimization on the following code.

$100.(\&, a, -, \$1)$	$103.(= [], \$1, \$2, u)$	$106.(= [], \$1, \$2, v)$
$101.(*, 4, i, \$2)$	$104.([] =, 0, x, \$1)$	$107.([] =, \$3, y, \$1)$
$102.(= [], \$1, \$2, \$3)$	$105.(= [], \$1, \$2, \$4)$	$108.(= [], \$1, \$2, w)$

34. Write the data flow equations for forward flow analysis of any path.
35. Write the data flow equations for backward flow analysis of any path.
36. Write the data flow equation for full path forward flow analysis.
37. Write the data flow equation for full path backward flow analysis.
38. What are the pessimistic and optimistic views on the solution of data flow equations?
39. Explain what operation will cancel the initialization of variables.
40. What is a available expression? What is a very busy expression?
41. What are the categories of uninitialized variable detection and initialized variable detection in data flow analysis?
42. What is reaching definition and UD chain?
43. What are the uses of UD chains?
44. What are UD Chain and DU Chain?
45. What is a dead variable? How to delete a dead variable?
46. What is loop-invariant expression?

Yanwei Zheng, Shandong University

47. What is the predecessor node of a loop?
48. What are the conditions for loop-invariant code motion?
49. What are induction variables and basic induction variables?
50. Fig. 8.3(a) contains uninitialized variables.
    (1) Perform live variable analysis and calculate the live variable set at the entry and exit of each basic block.
    (2) Perform uninitialized variable analysis and calculate the uninitialized variable set at the entry and exit of each basic block.
51. Fig. 8.3(b) contains common subexpressions.
    (1) Perform available expression analysis and calculate the available expression set at the entry and exit of each basic block.
    (2) Perform very busy expression analysis and calculate the very busy expression set at the entry and exit of each basic block.



(a) CFG containing uninitialized variables (b) CFG containing common subexpressions

Figure 8.3 CFG

Yanwei Zheng, Shandong University

52. Fig. 8.4 is a CFG for calculating the number of narcissus flowers.
    (1) Perform live variable analysis and calculate the live variable set at the entry and exit of each basic block.
    (2) Perform uninitialized variable analysis and calculate the uninitialized variable set at the entry and exit of each basic block.
    (3) Perform available expression analysis and calculate the available expression set at the entry and exit of each basic block.
    (4) Perform very busy expression analysis and calculate the very busy expression set at the entry and exit of each basic block.

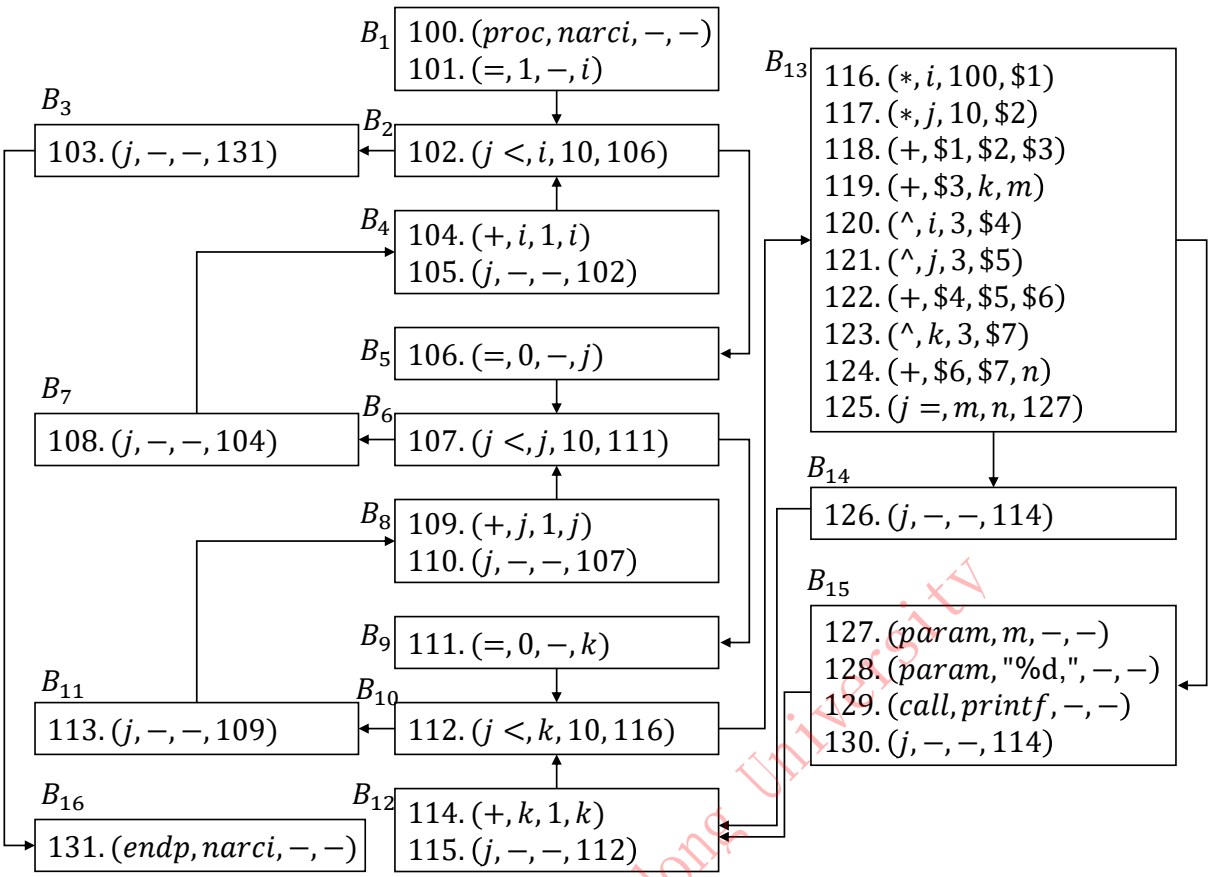
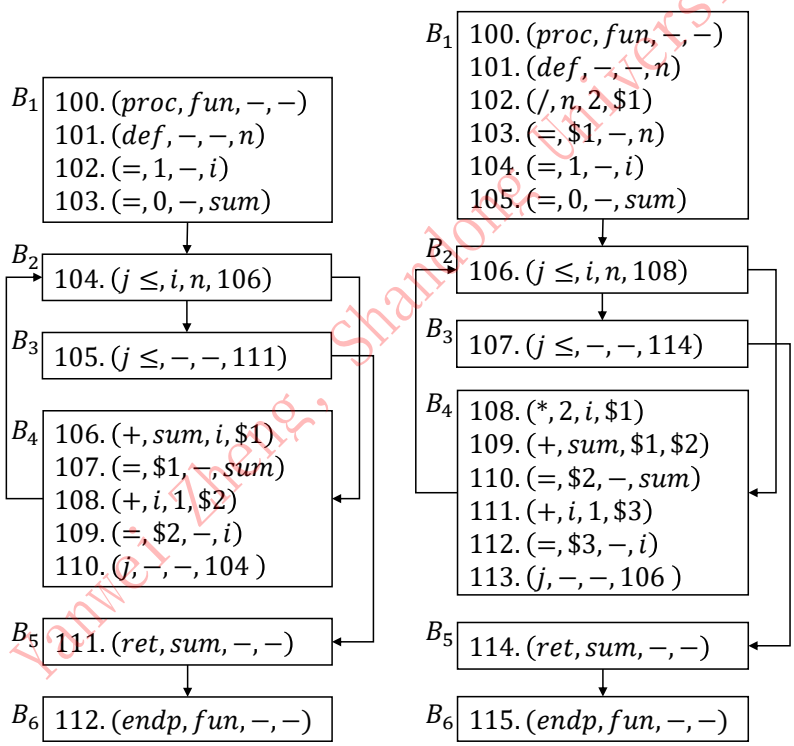


Figure 8.4 CFG for calculating the number of narcissus flowers

Yanwei Zheng Shandong University

53. Fig. 8.3(a) is a CFG.
    (1) Perform reaching definition analysis.
    (2) Calculate UD chains.
    (3) Perform live variable analysis with use site.
    (4) Calculate DU chains.
54. Fig. 8.3(b) is a CFG.
    (1) Perform reaching definition analysis.
    (2) Calculate UD chains.
    (3) Perform live variable analysis with use site.
    (4) Calculate DU chains.
55. Fig. 8.4 is a CFG.
    (1) Perform reaching definition analysis.
    (2) Calculate UD chains.
    (3) Perform live variable analysis with use site.
    (4) Calculate DU chains.
56. Fig. 8.5(a) is a CFG.
    (1) Perform reaching definition analysis.
    (2) Calculate UD chains.
    (3) Perform live variable analysis with use site.
    (4) Calculate DU chains.

57. Fig. 8.5(b) is a CFG.
    (1) Perform reaching definition analysis.
    (2) Calculate UD chains.
    (3) Perform live variable analysis with use site.
    (4) Calculate DU chains.
58. Fig. 8.5(a) is a CFG.
    (1) Perform copy propagation and constant folding.
    (2) Find the induction variables.
    (3) Delete the induction variables.
59. Fig. 8.5(b) is a CFG.
    (1) Perform copy propagation and constant folding.
    (2) Find the induction variables.
    (3) Perform strength reduction.
    (4) Delete the induction variables.



(a) Find the sum. (b) Find the sum of even numbers.

Figure 8.5 Single Loop.

Yanwei Zheng, Shandong University

60. Fig. 8.6 is a CFG.
    (1) Perform loop-invariant code motion.
    (2) Delete the common subexpressions.
    (3) Delete the dead assignment.
    (4) Perform copy propagation and constant folding.
61. Find the uninitialized variables for Fig. 8.3(a).
62. Delete the common subexpressions for Fig. 8.3(b).
63. Fig. 8.6 is a CFG.

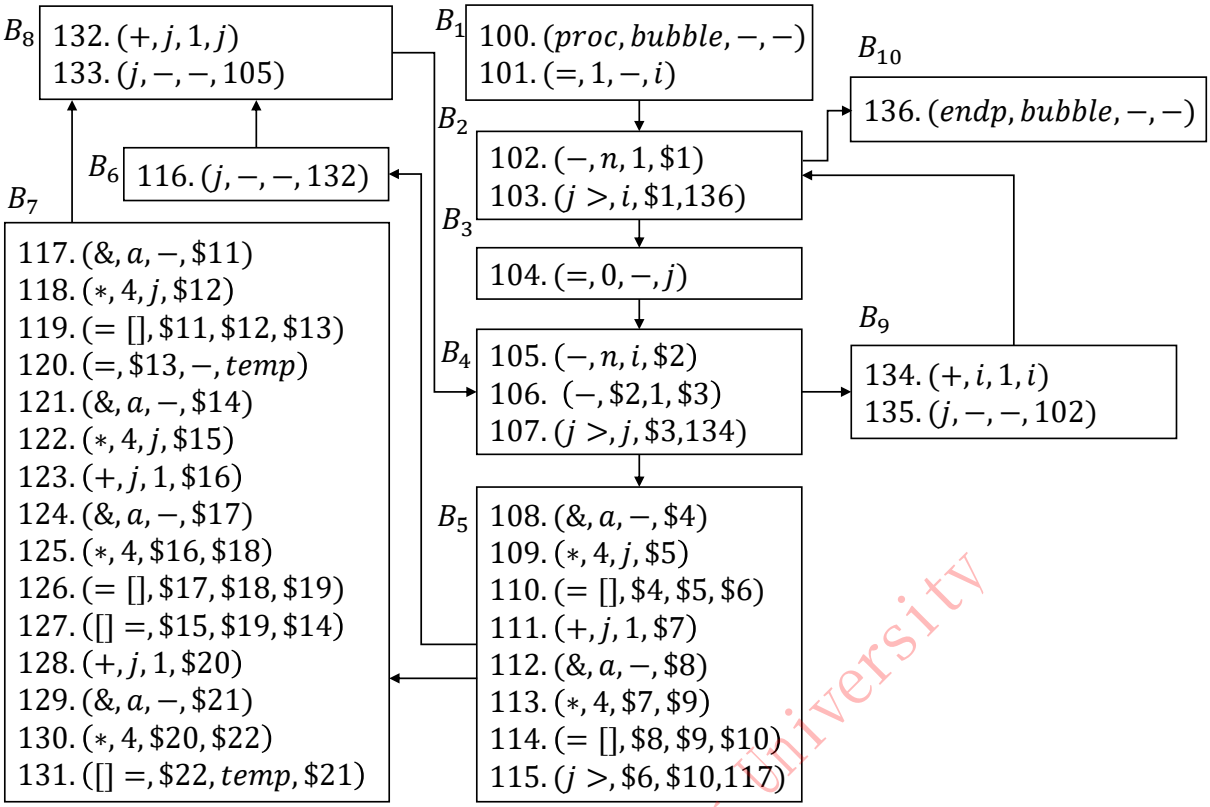


Figure 8.6 Bubble Sort.

(1) Perform loop-invariant code motion.
(2) Find the induction variables.
(3) Perform strength reduction.
(4) Delete the induction variables.

Yanwei Zheng, Shandong University

Chapter 9 Target Code Generation

Chapter 9 Exercise

1. Describe the three challenges faced in object code generation.
2. What factors affect instruction selection?
3. What are the considerations for improving code efficiency through instruction scheduling?
4. Describe the development history of register allocation.
5. What is next-use information? How to express it?
6. For the following intermediate code, assume that the available registers are R_0 and R_1, and u is the only live variable at the exit of the basic block.

$100.(-, x, y, \$1)$	$102.(+, b, 1, \$3)$	$104.(*, \$3, \$4, \$5)$
$101.(*, a, \$1, \$2)$	$103.(-, x, y, \$4)$	$105.(/, \$2, \$5, u)$

   (1) Calculate the next-use information and live information.
   (2) Generate target code using the simple code generation algorithm and write register descriptors and address descriptors during the code generation process.
7. For the following intermediate code, assume that the available registers are R_0 and R_1, and u is the only live variable at the exit of the basic block.

$100.(+, x, y, \$1)$	$102.(*, \$1, \$2, \$3)$	$104.(*, \$4, 2, \$5)$
$101.(-, x, y, \$2)$	$103.(=, \$3, -, \$4)$	$105.(=, \$5, -, u)$

   (1) Calculate the next-use information and live information.
   (2) Generate target code using the simple code generation algorithm and write register descriptors and address descriptors during the code generation process.
8. Write the target code corresponding to integer operations such as addition, subtraction, multiplication, division, and negation, as well as quadruple for assignment operations.
9. Write the target code corresponding to the float arithmetic quadruples.
10. Write the target code corresponding to the array arithmetic quadruples.
11. Write the target code corresponding to the transfer statement quadruples.
12. Write the target code corresponding to the address and pointer statement quadruples.
13. Write the target code corresponding to the type conversion statement quadruples.
14. Write the target code corresponding to the procedure quadruples.
15. Write the execution cost of the following instructions.

100.mov ebp, esp	102.mov [ebp + 8], eax	104.imul eax, ebx
101.mov eax, [ebp + 8]	103.add eax, ebx	105.sub eax, [ebp - 20]

16. Write the execution cost of the following instructions.

100.lea eax, [ebp - 20]	103.cmp eax, [ebp - 20]	106.jg L
101.inc eax	104.cmp [ebp - 20], ebx	107.push eax
102.inc [ebp - 20]	105.jmp L	108.pop ebp

17. Write the execution cost of the following instructions.

Yanwei Zheng, Shandong University

100.fld [ebp - 20]	103.fcomp [ebp - 20]	106.faddp st(0)
101.fld1	104.fnstsw ax	107.faddp [ebp - 20]
102.fabs	105.sahf	

18. For the following intermediate code, assume that the available registers are R_0 and R_1, and u is the only live variable at the exit of the basic block.

$100.(-, x, y, \$1)$	$102.(+, \$2, \$1, \$3)$	$104.(*, \$3, 2, \$5)$
$101.(*, \$1, z, \$2)$	$103.(+, \$1, \$3, \$4)$	$105.(*, \$4, \$5, u)$

(1) Generate target code using the simple code generation algorithm.
(2) Reorder the quadruples using DAG target code optimization algorithm.
(3) Generate target code using the simple code generation algorithm for the reordered quadruples.
(4) Compare the performance of the target code before and after optimization.

19. For the following intermediate code, assume that the available registers are R_0 and R_1, and u is the only live variable at the exit of the basic block.

$100.(+, x, y, \$1)$	$102.(*, \$1, 2, \$3)$	$104.(*, \$3, \$4, \$5)$
$101.(-, x, y, \$2)$	$103.(*, \$2, 2, \$4)$	$105.(-, \$5, 1, u)$

(1) Generate target code using the simple code generation algorithm.
(2) Reorder the quadruples using DAG target code optimization algorithm.
(3) Generate target code using the simple code generation algorithm for the reordered quadruples.
(4) Compare the performance of the target code before and after optimization.

20. For the following intermediate code, assume that the available registers are R_0 and R_1, and u is the only live variable at the exit of the basic block.

$100.(+, x, y, \$1)$	$102.(*, \$1, \$2, \$3)$	$104.(+, \$3, \$4, \$5)$
$101.(-, x, y, \$2)$	$103.(*, \$2, \$2, \$4)$	$105.(-, \$5, 1, u)$

(1) Optimize the intermediate code using DAG.
(2) Reorder the quadruples using DAG target code optimization algorithm.
(3) Generate target code using the simple code generation algorithm for the reordered quadruples.

21. For the following intermediate code, assume that the available registers are R_0 and R_1, and u and v are the live variables at the exit of the basic block.

$100.(+, x, 1, \$1)$	$103.(=, 1, -, x)$	$106.(+, \$3, \$4, u)$
$101.(-, x, 1, \$2)$	$104.(=, 2, -, y)$	$107.(+, x, 1, v)$
$102.(*, \$1, \$2, \$3)$	$105.(+, x, y, \$4)$	$108.(j <, u, v, 111)$

(1) Optimize the intermediate code using DAG.
(2) Reorder the quadruples using DAG target code optimization algorithm.
(3) Generate target code using the simple code generation algorithm for the reordered quadruples.

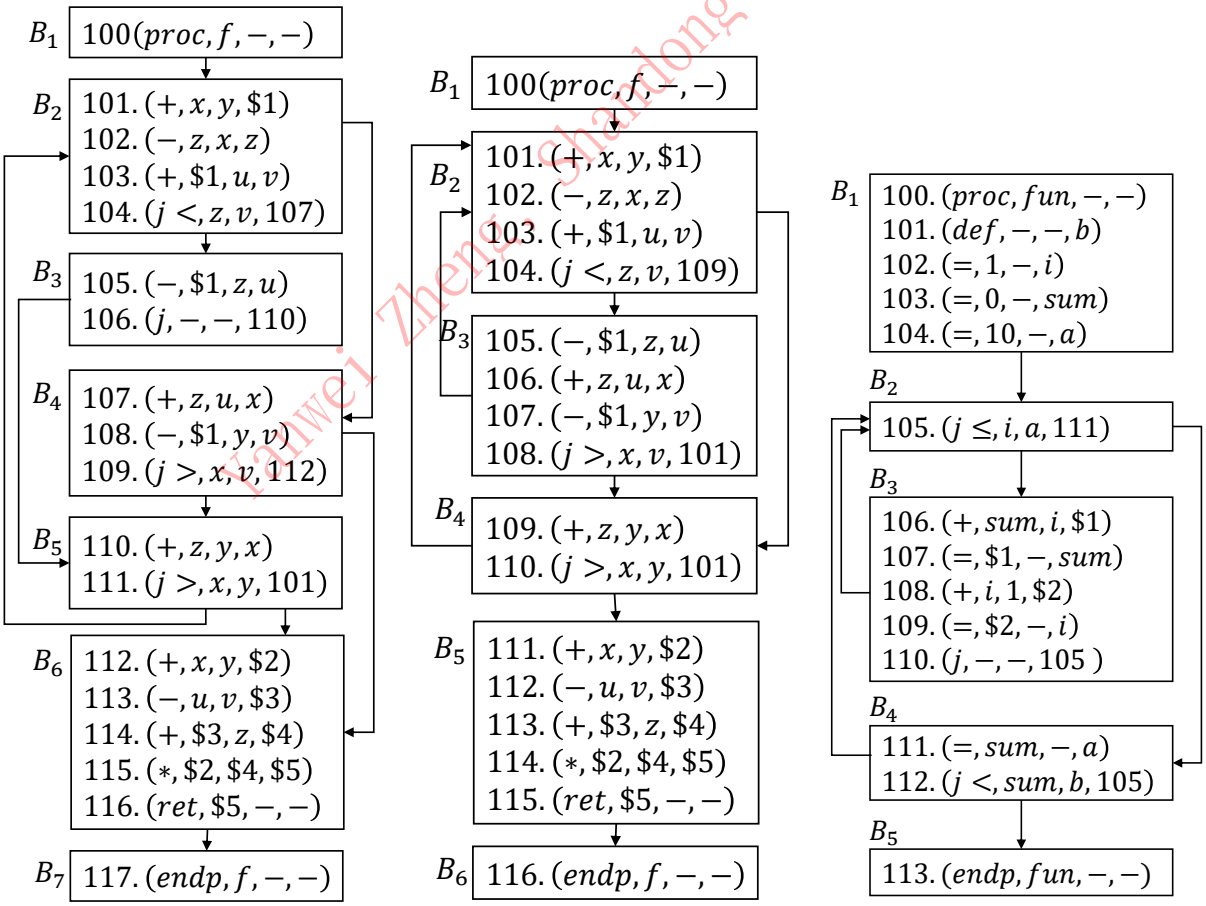
22. In the CFG shown in Fig. 9.1(a), two loops $B_2 \to B_3 \to B_5 \to B_2$ and $B_2 \to B_4 \to B_5 \to B_2$ have some same nodes. When allocating registers to variables in a loop, the execution cost saved by two loops is added together as the instruction cost saved after the fixed allocation of registers to the variables.

Yanwei Zhang, Shandong University

(1) Perform live variable analysis.
(2) Calculate the cost saved for each variable after fixed register allocation.
(3) Select three variables if the registers EAX, EBX, and EDX are fixedly allocated to three variables in the loops.
(4) Other registers, including ECX, ESI, and EDI, can be allocated to other variables. Generate target code using simple code generation algorithms.

23. In the CFG shown in Fig. 9.1(b), loop $B_2 \to B_3 \to B_2$ is the inner nesting loop of $B_2 \to B_3 \to B_4 \to B_2$ and $B_2 \to B_4 \to B_2$. Due to the more iterations in the inner loop, weighted summation is used to calculate saved execution costs when allocating registers to variables in the loop. For example, if there is n loops ($n = 3$ in this question), the saved cost of x is calculated by $cost(x) = \sum_{i=1}^{n} cost_i(x) \times 10^{d_i - 1}$, where $cost_i(x)$ is the saved cost of x by the ith loop, and d_i is the depth of this loop.
(1) Perform live variable analysis.
(2) Calculate the cost saved for each variable after fixed register allocation.
(3) Select three variables if the registers EAX, EBX, and EDX are fixedly allocated to three variables in the loops.
(4) Other registers, including ECX, ESI, and EDI, can be allocated to other variables. Generate target code using simple code generation algorithms.

Yanwei Zheng, Shandong University



(a) CFG of multiple loops. (b) CFG of nesting loops 1. (c) CFG of nesting loops 2.

Figure 9.1 CFG of Loop register allocation.

24. In the CFG shown in Fig. 9.1(c), loop $B_2 \rightarrow B_3 \rightarrow B_2$ is the inner nesting loop of $B_2 \rightarrow B_4 \rightarrow B_2$. Due to the more iterations in the inner loop, weighted summation is used to calculate saved execution costs when allocating registers to variables in the loop. For example, if there is n loops ($n = 2$ in this question), the saved cost of x is calculated by $cost(x) = \sum_{i=1}^{n} cost_i(x) \times 10^{d_i - 1}$, where $cost_i(x)$ is the saved cost of x by the ith loop, and d_i is the depth of this loop.
    (1) Perform live variable analysis.
    (2) Calculate the cost saved for each variable after fixed register allocation.
    (3) Select three variables if the registers EAX, EBX, and EDX are fixedly allocated to three variables in the loops.
    (4) Other registers, including ECX, ESI, and EDI, can be allocated to other variables. Generate target code using simple code generation algorithms.
25. In DAG, the attachment of a node is its definition variable. Node n_2 is parent node of n_1, indicating that the definition statement of n_2 will use the variable n_1. Based on these information, design an algorithm using DAG to calculate the next-use information for intermediate code.
26. C printf scanf *cstr* $(param, cstr, -, -)$ *cstr* \r \n \t 0Dh 0Ah 09h first parameter is a constant string when calling printf and scanf functions of the C runtime library. Assume the parameter is *cstr*, then it will generate the quadruple $(param, cstr, -, -)$. In the constant string *cstr* of this instruction, the escape characters entered by the user, such as \r, \n and \t, have been converted by the lexical analyzer to the real characters 0Dh, 0Ah and 09h, respectively. Suppose there is no character 0 indicating the end of the last string.
    (1) Design a stack pushing algorithm for *cstr*, which requires aligning bytes to multiples of 4.
    (2) *cstr* After the calling is completed,design the stack-frame clean-up algorithm of *cstr*(involving modifications to the entire procedure call algorithm).
27. Draw a figure to illustrate the framework of the graph-coloring register allocation algorithm and briefly describe the functions of each part.
28. What is the purpose of allocating virtual registers?
29. Briefly describe the data items and their meanings in low-level intermediate code representation.
30. What is a register conflict graph? What is the construction method?
31. How to handle the combination of EDX:EAX registers in x86 division instruction during the construction of register conflict diagrams?
32. What is merging virtual registers? How to operate it specifically?
33. What is the role of adjacency table in graph coloring register allocation? Describe its data structure briefly.
34. What is the spilling cost in graph coloring register allocation? What aspects are included?
35. What is pruning conflict graph in graph coloring register allocation? Describe its rules briefly.
36. What is register pressure? How to reduce register pressure?
37. There are 4 registers of EAX, EBX, ECX and EDX in x86. Fig. 9.1(b) shows a CFG.
    (1) Allocate registers using graph-coloring register allocation algorithm.
    (2) Generate target code based on the register allocation results.
38. There are 4 registers of EAX, EBX, ECX and EDX in x86. Fig. 9.1(c) shows a CFG.
    (1) Allocate registers using graph-coloring register allocation algorithm.
    (2) Generate target code based on the register allocation results.

Yanwei Zheng Shandong University

39. There are 6 registers in x86. Fig. 9.2 shows a CFG.
    (1) Allocate registers using graph-coloring register allocation algorithm.
    (2) Generate target code based on the register allocation results.

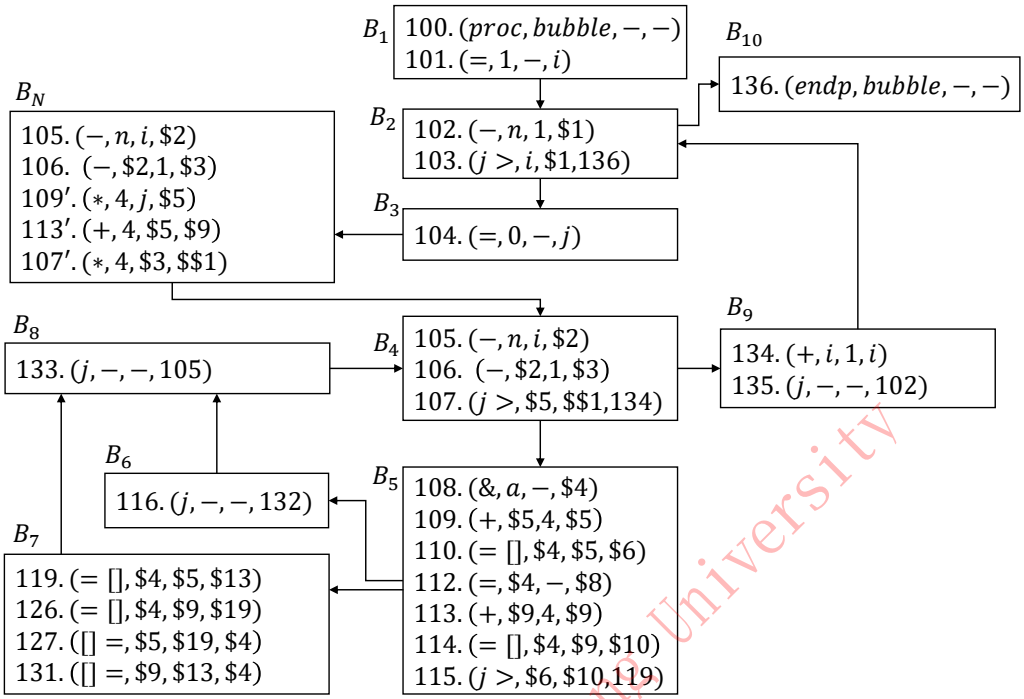


Figure 9.2 CFG after constant folding and induction variable deleting.

40. Try to improve the spilling algorithm for graph-coloring register allocation. If an spilling variable is used multiple times in the same basic block and is not killed before the last use in the block, this value only needs to be loaded once in this basic block.
41. What is CFG linearization? Describe its rules briefly.
42. What is the live range?
43. There are 4 registers of EAX, EBX, ECX and EDX in x86. Fig. 9.1(b) shows a CFG.
    (1) Allocate registers using linear scan register allocation algorithm.
    (2) Generate target code based on the register allocation results.
44. There are 4 registers of EAX, EBX, ECX and EDX in x86. Fig. 9.1(c) shows a CFG.
    (1) Allocate registers using linear scan register allocation algorithm.
    (2) Generate target code based on the register allocation results.
45. There are 6 registers in x86. Fig. 9.2 shows a CFG.
    (1) Allocate registers using linear scan register allocation algorithm.
    (2) Generate target code based on the register allocation results.
46. The global target code generation algorithm for constructing type conversion instructions, it is needed to support the case where the operands are constant.
47. Write the optimized code for peephole as follows.

100. add eax, 0	103. add eax, ebx	106. pop ebp
101. cmp -1, 0	104. L1:	107. L2: ...
102. jl L2	105. imul eax, 1	

48. Write the optimized code for peephole as follows.

Yanwei Zheng, Shandong University

100. cmp eax, ebx
101. jl L1
102. jmp L2
103. L1:
104. add ecx, edx
105. imul eax, 1
106. pop ebp
107. L2: ...

49. Sometimes we may encounter a situation where a value is stored to the memory and immediately loaded to a register. For example, the instruction "mov a, R_0" is immediately followed by an instruction "mov R_1, a", where registers R_0 and R_1 may or may not be the same register. Please design an algorithm that optimizes this pair of instructions through peephole optimization.

50. The shift instructions calculate much faster than the multiplication instructions. If the multiplier is a constant, such as "imul $R_0, 70$", since $70 = 2^6 + 2^2 + 2^1$, the multiplication instruction can be modified to the following 6 instructions.

1. mov R_1, R_0
2. shl $R_0, 1$
3. shl $R_1, 2$
4. add R_0, R_1
5. shl $R_1, 4$
6. add R_0, R_1

The idea is to use another register R_1 to assist in calculation. The lowest bit left shifted by 1 in R_0, whereas the remaining left shifted by 2 or 6 (2 bits left and 4 bits left) in R_1, and accumulated to R_0. In general systems, if a constant can be decomposed into a 2-power sum of no more than 3 terms, this optimization is effective. The limitation of this optimization is that unsigned multiplication is computed using signed multiplication instructions, and multiplication does not overflow to make the highest bit be 1. Design this optimization algorithm. (Hint: Write 70 as a binary number, with its 1st, 2nd, and 6th bits being 1 and the others being 0; or divide continuously by 2 to find the remainder, starting from 0, with the remainder of the 1st, 2nd, and 6th division by 2 being 1.)

Yanwei Zheng, Shandong University

Chapter 10 Translation of Object Oriented Languages

Chapter 10 Exercise

1. Describe the rules for member variable access in object-oriented language.
2. Describe the rules for method invocation in object-oriented language.
3. What is a class hierarchy table? What is the similarity and difference between it and display tables in Pascal?
4. For code 10.1, draw the runtime class hierarchy table and a schematic diagram of the arrangement of each member variable in memory.

Code 10.1 Class Inheritance

```
public A : Object {
  int x, y;
  A(int x, int y) { this.x = x; this.y = y; }
  void f1(int a, int b) {...}
  double f2(double a, double b) {...}
}
public B : A {
  double sum;
  B(double sum) : A(0, 100) { this.sum = sum; }
  double f2(double a, double b) {...}
}
```

5. How symbol tables are organized in object-oriented language?
6. Describe the rules for searching symbol tables in various situations.
7. Draw the class symbol table for code 10.1.
8. Draw the class method table for code 10.1.
9. Please design a reasonable runtime storage and access strategy for static properties.
10. Describe the various situations and rules of object assignment.
11. Describe the various scenarios of type checking and illustrate their execution logic with a diagram.
12. What is a collection of related class methods? How to calculate it?
13. Describe the execution logic of attribute reference $id1.id2$ in single inheritance.
14. Describe the execution logic of method calling $id1.id2()$ in single inheritance.
15. Describe the execution logic of method calling $id1 :: id2()$ in single inheritance.
16. Translate the declaration statement of code 10.1 and fill in the symbol table.
17. For code 10.1, there is declaration code "A a1, a2; B b;", translate the code "a1 = new A(30, 40); a2 = a1; free a2;".
18. For code 10.1, there is declaration code "A a; B b;", translate the code "if (a.typeof(B)) b = a;".
19. For code 10.1, there is declaration code "A a; B b;", translate the code "if (typeid(a) = typeid(B)) b = a;".
20. For code 10.1, there is declaration code "A a; B b, double t;", translate the code "a.f1(3, 4); t = b.f2(1.0, 2.0);".

Yanwei Zhang, Shandong University